



Transformers

DL4DS – Spring 2026

A Brief History of Transformers



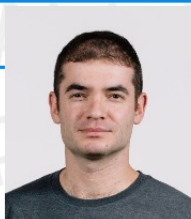
2000

Yoshua Bengio*



2014

Ilya Sutskever*



2014

Dzmitry Bahdanau*



2017

A Team at Google



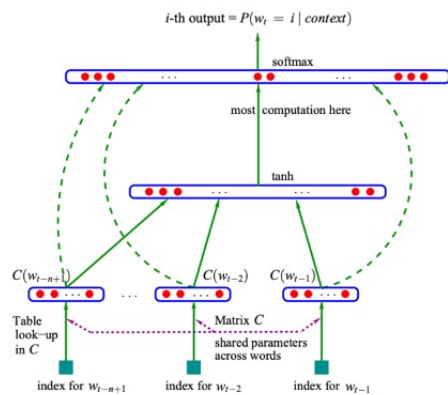
Use LSTMs

Add Attention

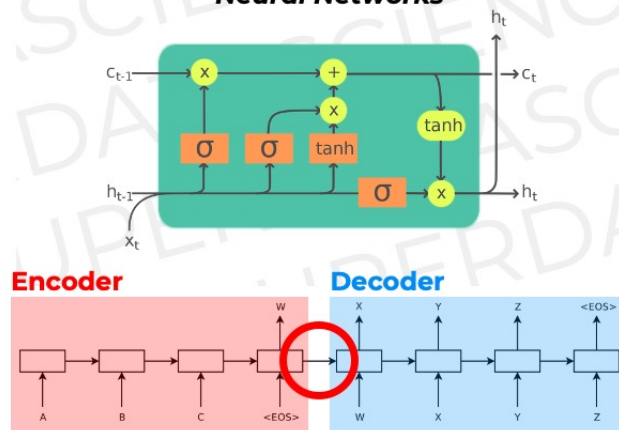
Remove LSTMs

Attention is all you need

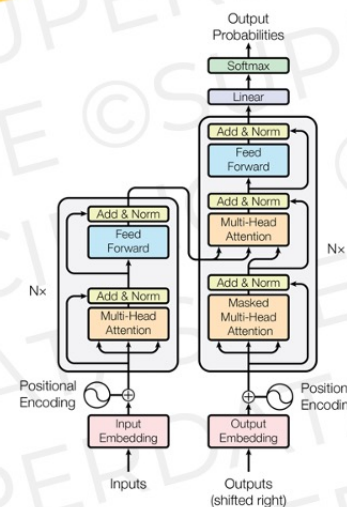
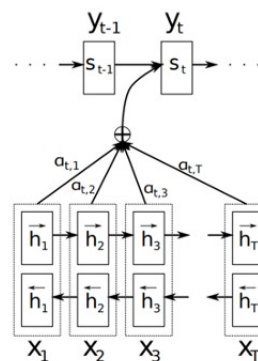
A Neural Probabilistic Language Model



Seq-to-Seq Learning with Neural Networks



Neural Machine Translation by Jointly Learning to Align and Translate



*And others; Chronological analysis inspired by Andrej Karpathy's lecture, [youtube.com/watch?v=XfpMkf4rD6E](https://www.youtube.com/watch?v=XfpMkf4rD6E)



A Neural Probabilistic Language Model

Bengio et al, 2000 and 2003

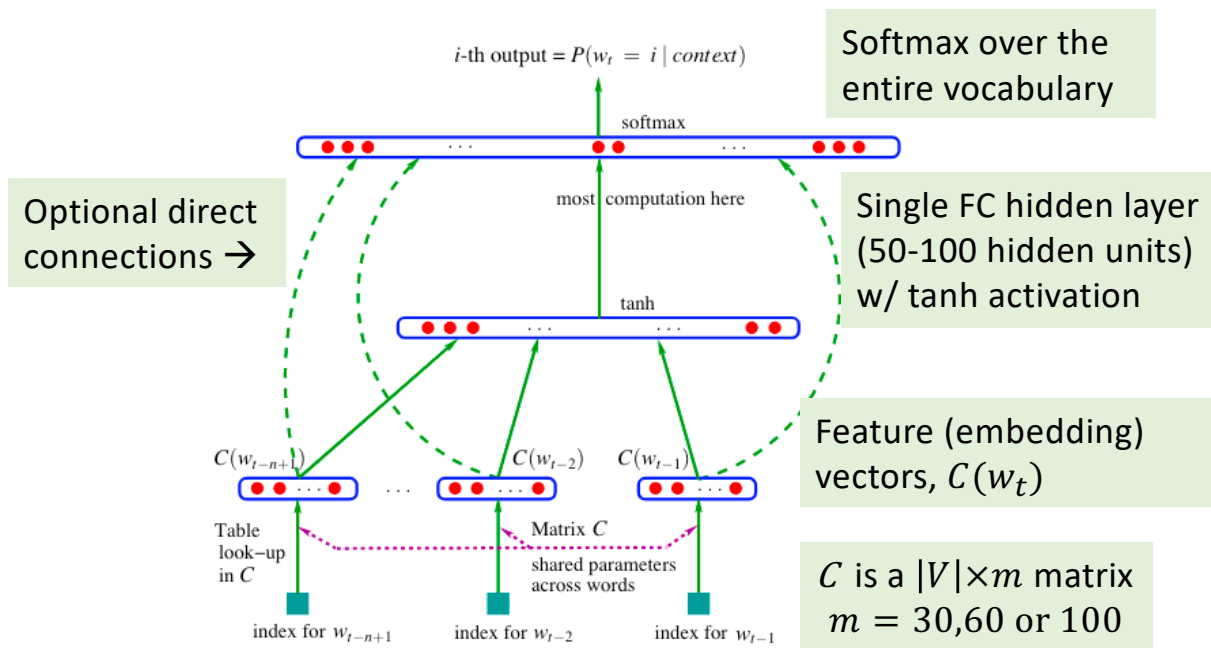


Figure 1: Neural architecture: $f(i, w_{t-1}, \dots, w_{t-n+1}) = g(i, C(w_{t-1}), \dots, C(w_{t-n+1}))$ where g is the neural network and $C(i)$ is the i -th word feature vector.

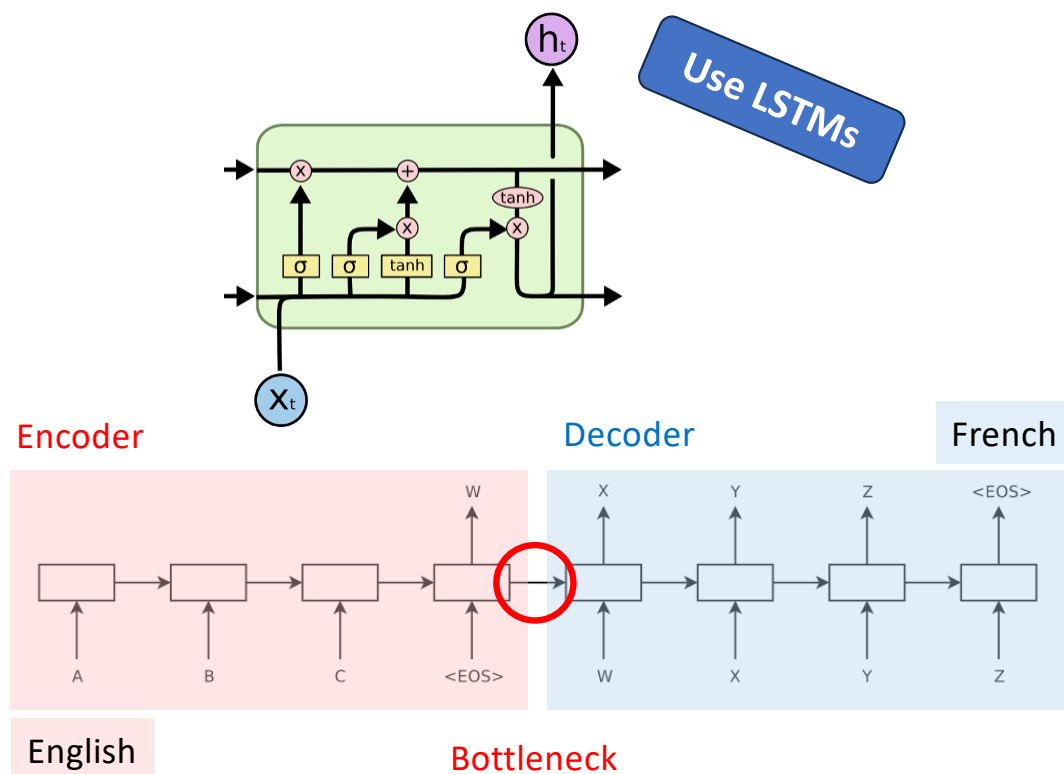
$w_t \in V$ words in the vocabulary ~16k-18k

- Build a probabilistic language model from NNs
- Feed forward network with shared learned parameters, C , that create embeddings
- Precursor to Word2Vec
- Predicts the probability of a word at time t , based on the context of the last n words
- 13-14M parameters

Limited to context length of $n \in \{3, 4, 5, 6\}$. Tiny!!

Sequence to Sequence Learning with Neural Networks

Sutskever et al (2014)

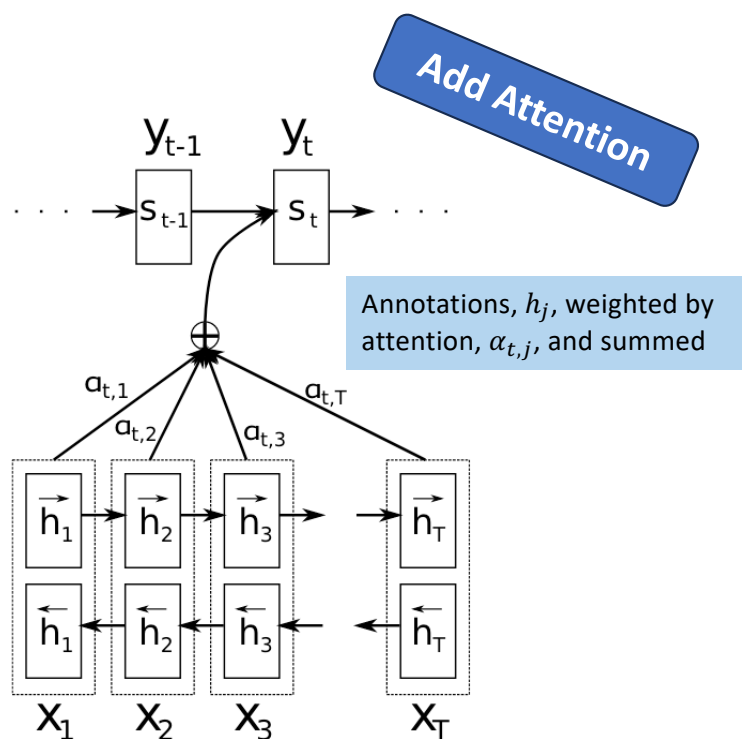


- Used 4-layer LSTMs in an Encoder/Decoder structure
- Used 1000-D embeddings learned in training w/ 160K source vocabulary
- Estimate the probability of $p(y_1, \dots, y_{T'} | x_1, \dots, x_T)$ where $T' \neq T$, where T, T' was $\sim 20-30$
- Encoder mapped sequence to a fixed size token (1000-D hidden state)
- The hidden state may not encode all the information needed by the decoder
- $\sim 380M$ parameters

Bottleneck between Encoder and Decoder!

Neural Machine Translation by Jointly Learning to Align and Translate

Bahdanau, Cho & Bengio (2014-15)



$T = 30$ or 50

- Used bi-directional LSTMs
- Concatenated forward and backward hidden states called “annotations” Type equation here.
- Automatically “soft-search” parts of input that influence the output
- Overcomes the bottleneck of a fixed size hidden state between encoder and decoder
- Significantly improved ability to comprehend longer sequences

Neural Machine Translation by Jointly Learning to Align and Translate

Bahdanau, Cho & Bengio (2014-15)

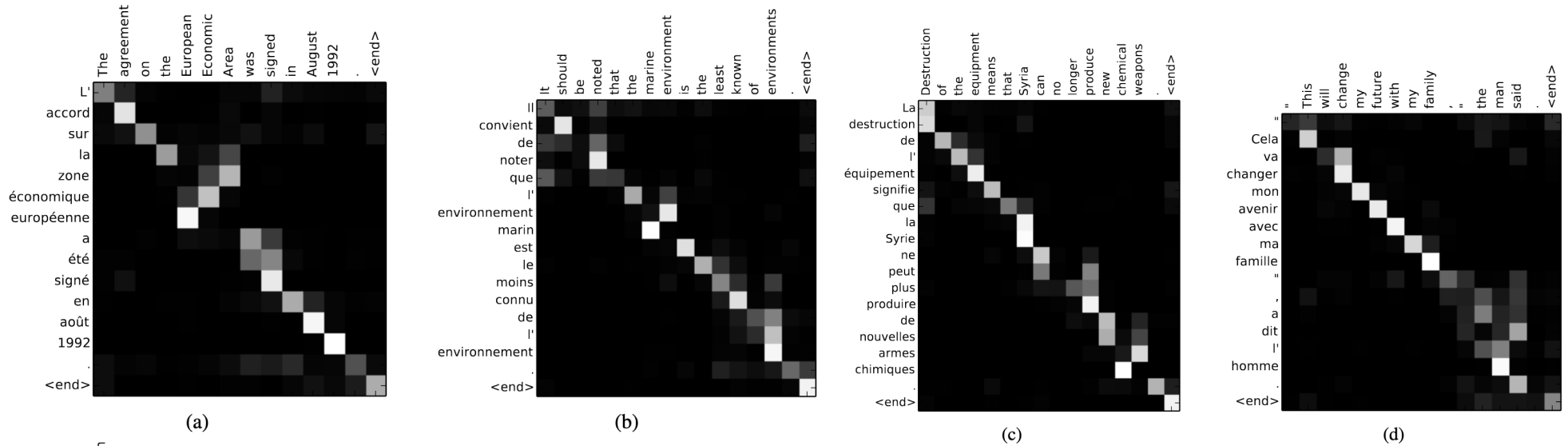
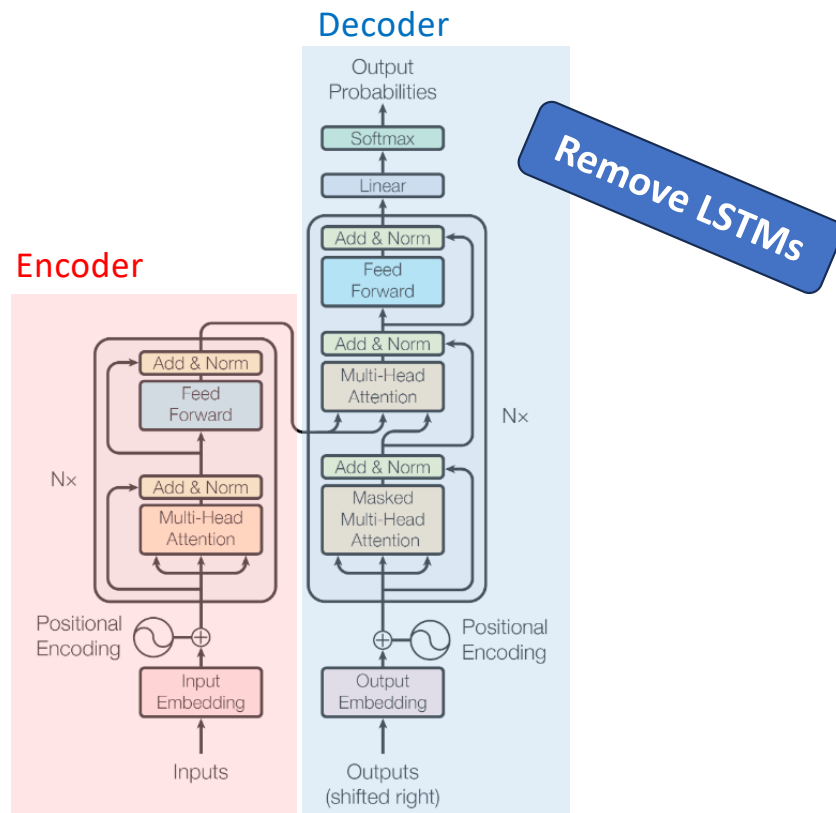


Figure 3: Four sample alignments found by RNNsearch-50. The x-axis and y-axis of each plot correspond to the words in the source sentence (English) and the generated translation (French), respectively. Each pixel shows the weight α_{ij} of the annotation of the j -th source word for the i -th target word (see Eq. (6)), in grayscale (0: black, 1: white). (a) an arbitrary sentence. (b–d) three randomly selected samples among the sentences without any unknown words and of length between 10 and 20 words from the test set.

Attention is All You Need

Vaswani et al (2017)



- Removed LSTMs and didn't use convolutions
- Only attention mechanisms and MLPs
- Parallelizable by removing sequential hidden state computation
- Outperformed all previous models

Transformers applied to many NLP applications

- Translation
- Question answering
- Summarizing
- Generating new text
- Correcting spelling and grammar
- Finding entities
- Classifying bodies of text
- Changing style etc.

Transformers

- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models

Transformers

- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models

Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

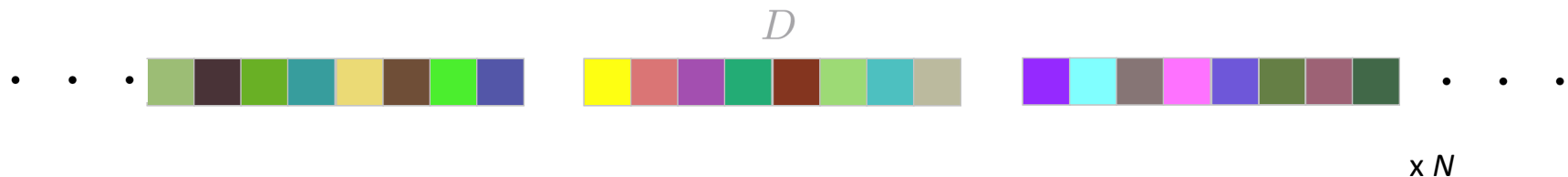


- Create a vocabulary of words (or word parts)
- Encode to a D -dimensional embedding vector.
- We'll look at tokenization and embedding encoding later.
- For now, assume a word is a token.

Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

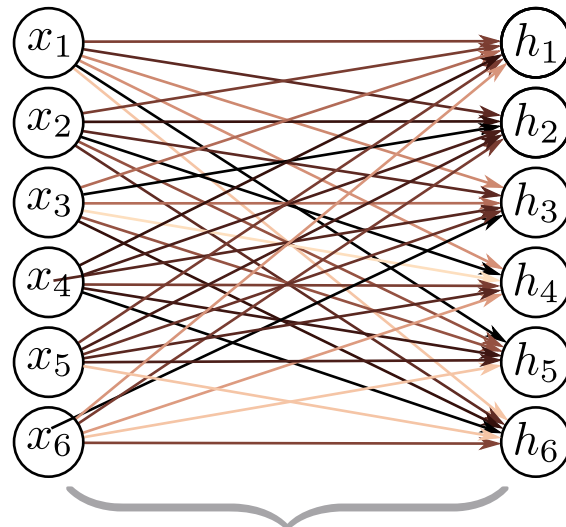


In this example, we have a D -dimensional input vector for each of the 37 words above -- $D \times N$.

Normally we would represent punctuation, capitalization, spaces, etc. as well.

Standard fully-connected layer

$$\mathbf{h} = \mathbf{a}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}]$$



Φ contains
 D^2 connections

Assuming D inputs and
 D hidden units.

Standard fully-connected layer

$$\mathbf{h} = \mathbf{a}[\boldsymbol{\beta} + \boldsymbol{\Omega}\mathbf{x}]$$

Problem:

- token (word) vectors may be 512 or 1024 dimensional
- need to process large segment of text
- Hence, would require a very large number of parameters
- Can't cope with text of different lengths

Conclusion:

- We need a model where parameters don't increase with input length

Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

The word **their** must “attend to” the word **restaurant**.

Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

The word **their** must “attend to” the word **restaurant**.

Conclusions:

- There must be connections between the words.
- The strength of these connections will depend on the words themselves.

Motivation

- Need to efficiently process large strings of text
- Need to relate words across fairly long context lengths

Self-Attention addresses these problems

Transformers

- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models

Dot-Product Self-Attention

1. Shares parameters to cope with long input passages of different lengths
2. Contains connections between word representations that depend on the words themselves

Dot-product self attention

- Takes N inputs of size $D \times 1$ and returns N inputs of size $D \times 1$
- Computes N **values** (no ReLU), for $n = 0, \dots, N - 1$.

$$\mathbf{v}_n = \beta_v + \Omega_v \mathbf{x}_n$$

Dot-product self attention

- Takes N inputs of size $D \times 1$ and returns N inputs of size $D \times 1$
- Computes N **values** (no ReLU)

$$\mathbf{v}_n = \beta_v + \Omega_v \mathbf{x}_n$$

- N outputs are weighted sums of these values

$$\mathbf{sa}[\mathbf{x}_n] = \sum_{m=1}^N a[\mathbf{x}_n, \mathbf{x}_m] \mathbf{v}_m$$

Dot-product self attention

- Takes N inputs of size $D \times 1$ and returns N outputs of size $D \times 1$
- Computes N **values** (no ReLU)

$$\mathbf{v}_n = \beta_v + \Omega_v \mathbf{x}_n$$

- N outputs are weighted sums of these values

'sa' is the self-attention weight for the n^{th} output of the sequence $x_1, \dots, x_N$.

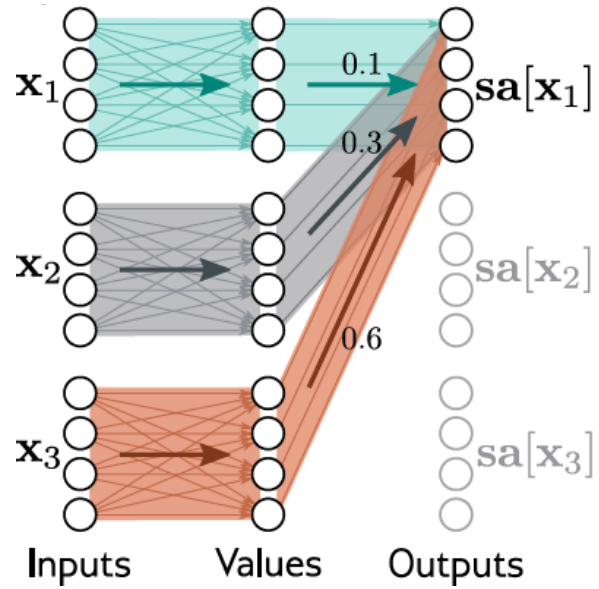
$$\mathbf{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N] = \sum_{m=1}^N a[\mathbf{x}_m, \mathbf{x}_n] \mathbf{v}_m.$$

Scalar self-attention weights that represent how much attention the n^{th} token should pay to the m^{th} token

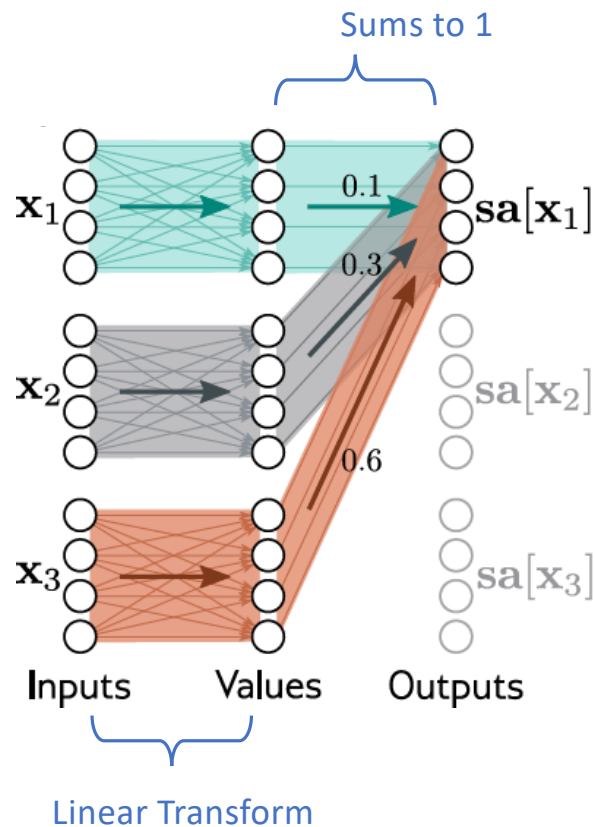
- Weights depend on the inputs themselves

$a[\cdot, \mathbf{x}_n]$ are non-negative and sum to one

Attention as routing



Attention as routing



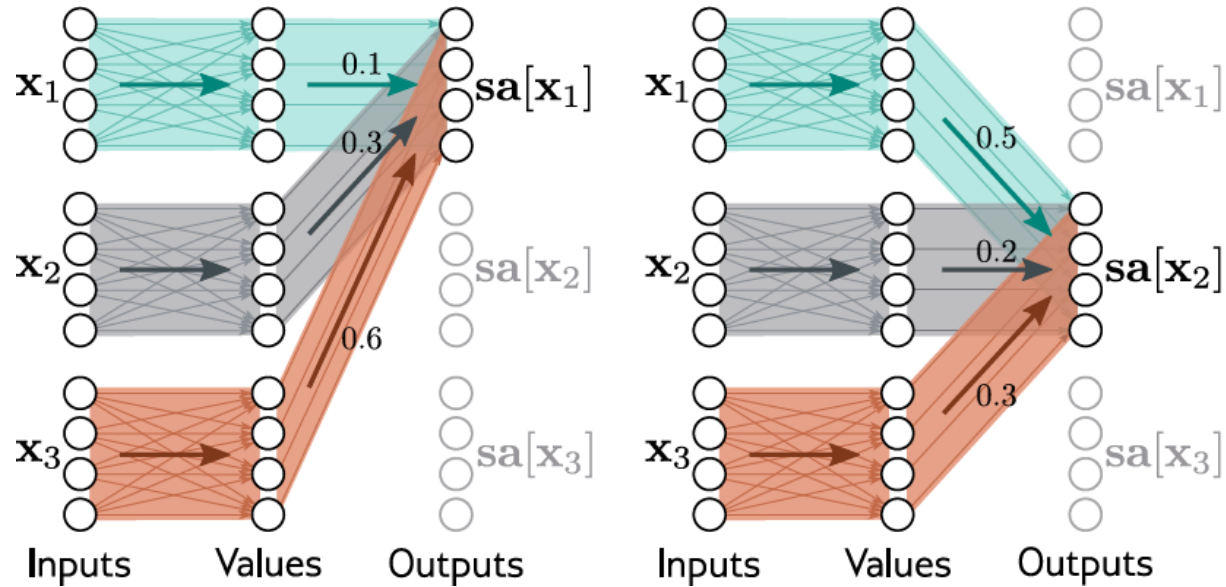
Here:

of inputs, $N = 3$

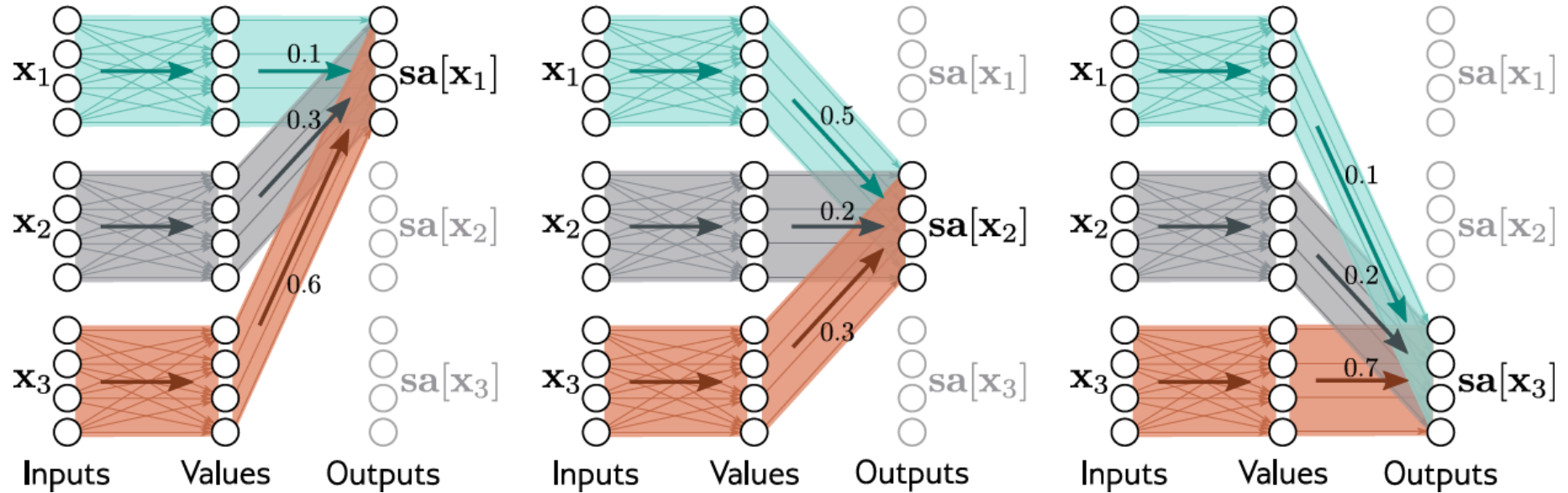
Dimension of each input, $D = 4$

We'll show how to calculate the self-attention weights shortly.

Attention as routing



Attention as routing



Attention weights

- Compute N “queries” and N “keys” from input

$$\mathbf{q}_n = \beta_q + \Omega_q \mathbf{x}_n$$
$$\mathbf{k}_n = \beta_k + \Omega_k \mathbf{x}_n,$$

- Calculate similarity and pass through softmax:

$$a[\mathbf{x}_n, \mathbf{x}_m] = \text{softmax}_m [\text{sim}[\mathbf{k}_m \mathbf{q}_n]]$$
$$= \frac{\exp [\text{sim}[\mathbf{k}_m \mathbf{q}_n]]}{\sum_{m'=1}^N \exp [\text{sim}[\mathbf{k}'_{m'} \mathbf{q}_n]]},$$

Attention weights

- Compute N “queries” and N “keys” from input

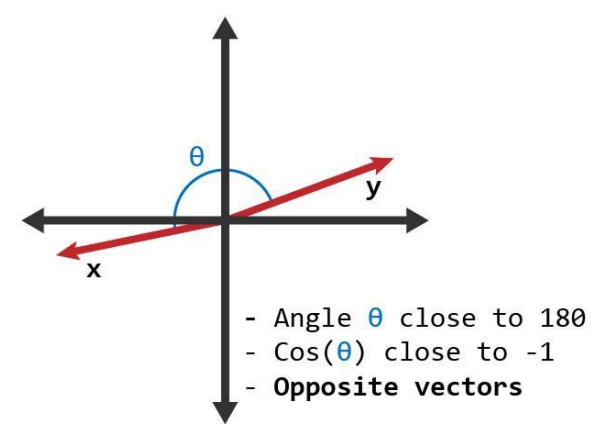
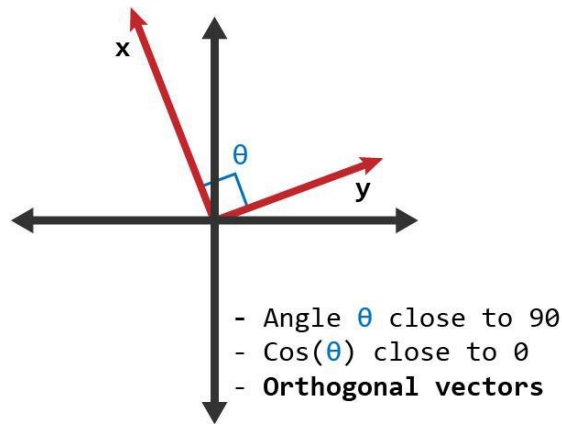
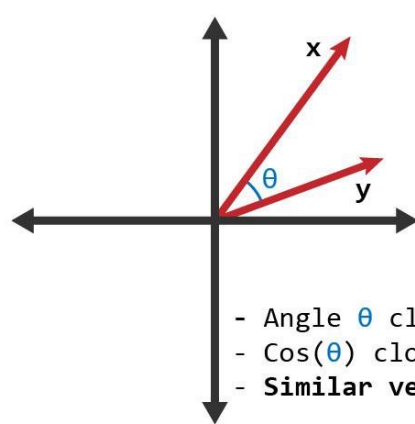
$$\mathbf{q}_n = \beta_q + \Omega_q \mathbf{x}_n$$
$$\mathbf{k}_n = \beta_k + \Omega_k \mathbf{x}_n,$$

- Take dot products and pass through softmax:

$$a[\mathbf{x}_n, \mathbf{x}_m] = \text{softmax}_m [\mathbf{k}_m^T \mathbf{q}_n]$$
$$= \frac{\exp [\mathbf{k}_m^T \mathbf{q}_n]}{\sum_{m'=1}^N \exp [\mathbf{k}_{m'}^T \mathbf{q}_n]}$$

Dot product = measure of similarity

$$\mathbf{x}^T \mathbf{y} = |\mathbf{x}| |\mathbf{y}| \cos(\theta)$$



A drawback of the dot product as similarity measure is the magnitude of each vector influences the value. More rigorous to divide by magnitudes.

Cosine Similarity: $\frac{\mathbf{x}^T \mathbf{y}}{|\mathbf{x}| |\mathbf{y}|} = \cos(\theta)$

Motivation

Design neural network to encode and process text:

The restaurant refused to serve me a ham sandwich, because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambience was just as good as the food and service.

Conclusions:

- ✓ We need a model where parameters don't increase with input length, e.g.

$$\phi = \{\beta_v, \Omega_v, \beta_q, \Omega_q, \beta_k, \Omega_k\}$$

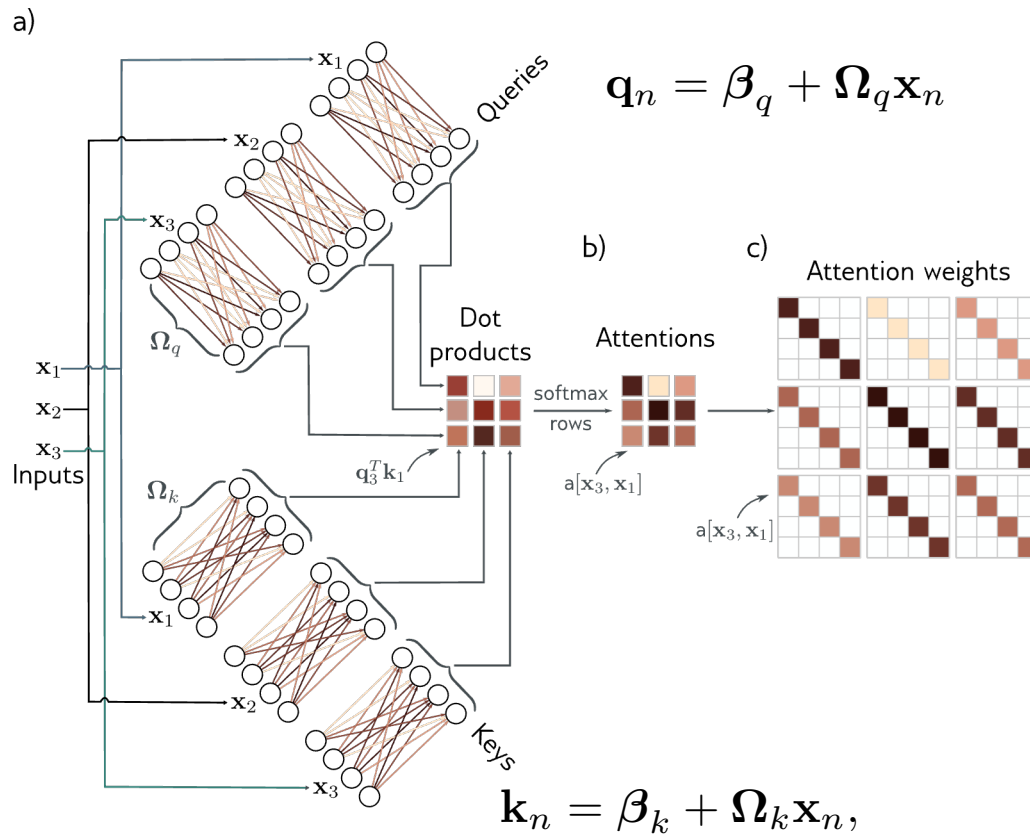
- ✓ There must be connections between the words.
- ✓ The strength of these connections will depend on the words themselves.

Ok, we defined *queries*, *keys* and *values*, but how are they used?

Transformers

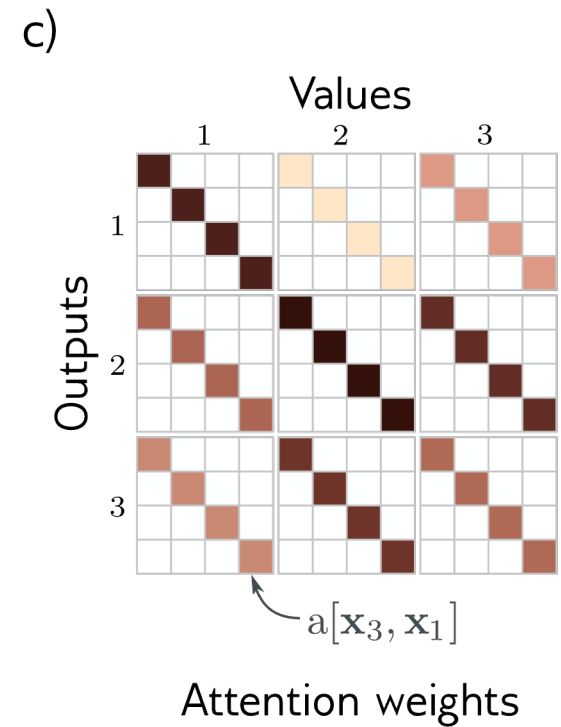
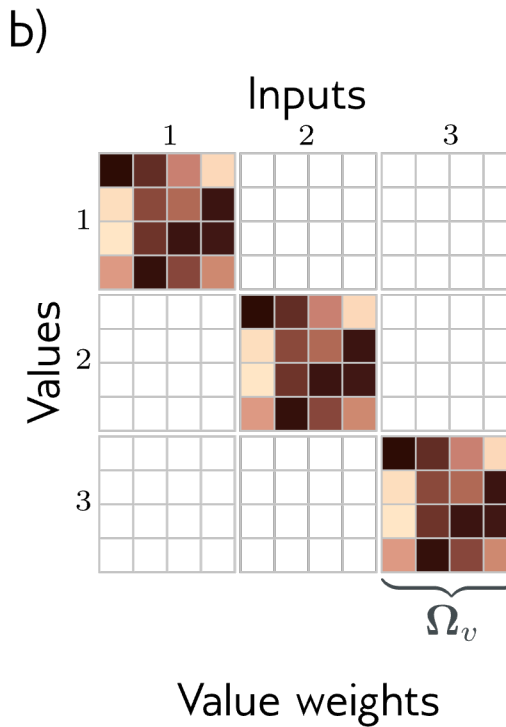
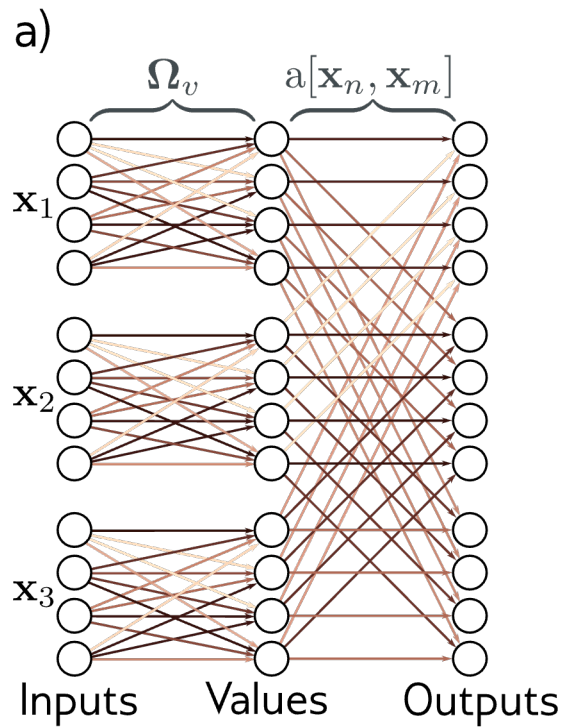
- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models

Computing Attention Weights



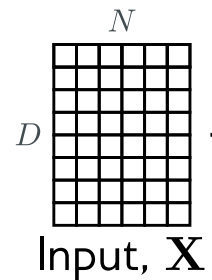
$$a[\mathbf{x}_n, \mathbf{x}_m] = \text{softmax}_m [\mathbf{k}_m^T \mathbf{q}_n]$$

Computing Values and Self-Attention Outputs as Sparse Matrix Ops



From Input Vector to Input Matrix

- Store N input vectors in matrix X



- Compute values, queries and keys:

$$\mathbf{V}[\mathbf{X}] = \beta_v \mathbf{1}^T + \Omega_v \mathbf{X}$$

$$\mathbf{Q}[\mathbf{X}] = \beta_q \mathbf{1}^T + \Omega_q \mathbf{X}$$

$$\mathbf{K}[\mathbf{X}] = \beta_k \mathbf{1}^T + \Omega_k \mathbf{X},$$

- Combine self-attentions

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V}[\mathbf{X}] \cdot \mathbf{Softmax}[\mathbf{K}[\mathbf{X}]^T \mathbf{Q}[\mathbf{X}]] = \mathbf{V} \cdot \mathbf{Softmax}[\mathbf{K}^T \mathbf{Q}]$$

Scaled Dot Product Self-Attention

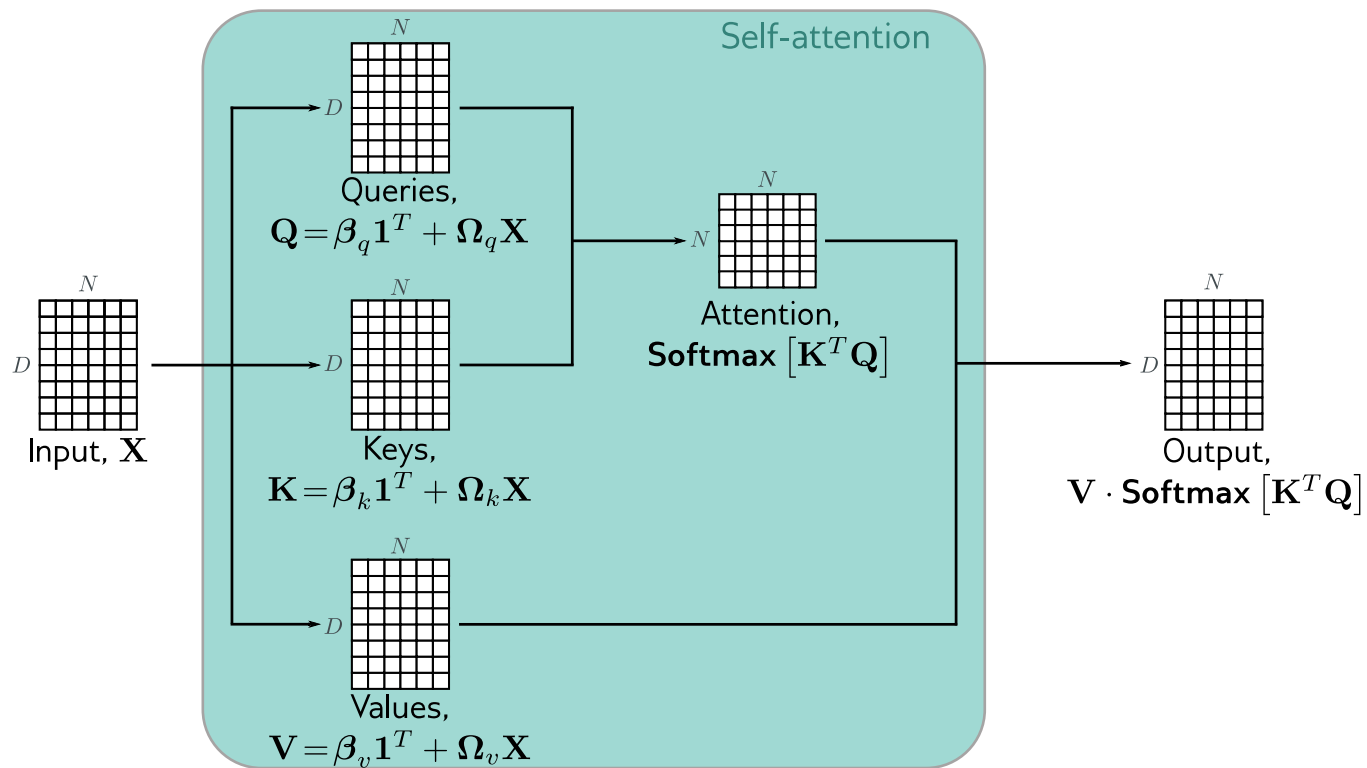
- To avoid the case where a large value dominates the softmax in

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q}]$$

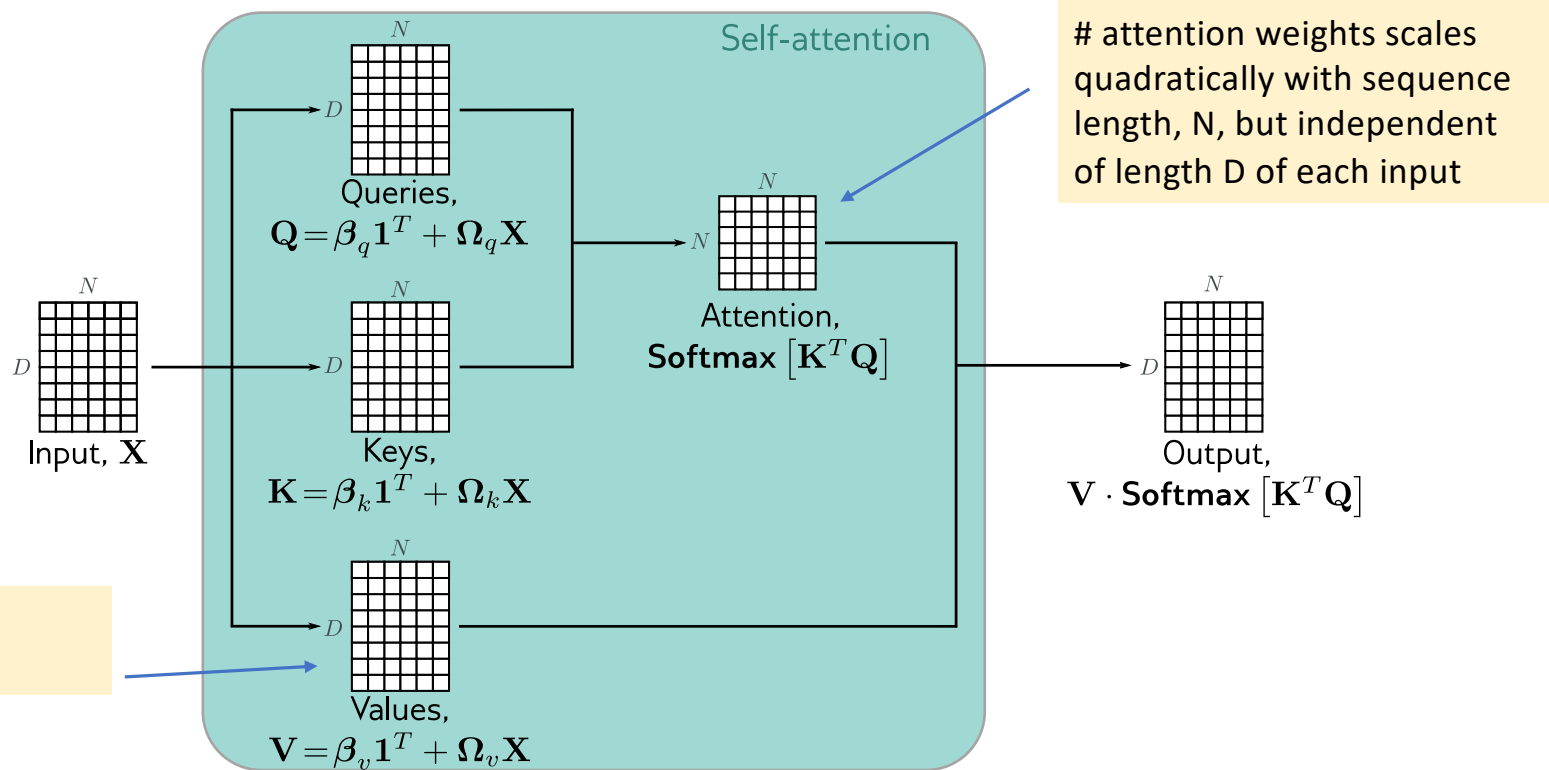
- you can scale the dot product by the square root of the dimension of the query

$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax} \left[\frac{\mathbf{K}^T \mathbf{Q}}{\sqrt{D_q}} \right]$$

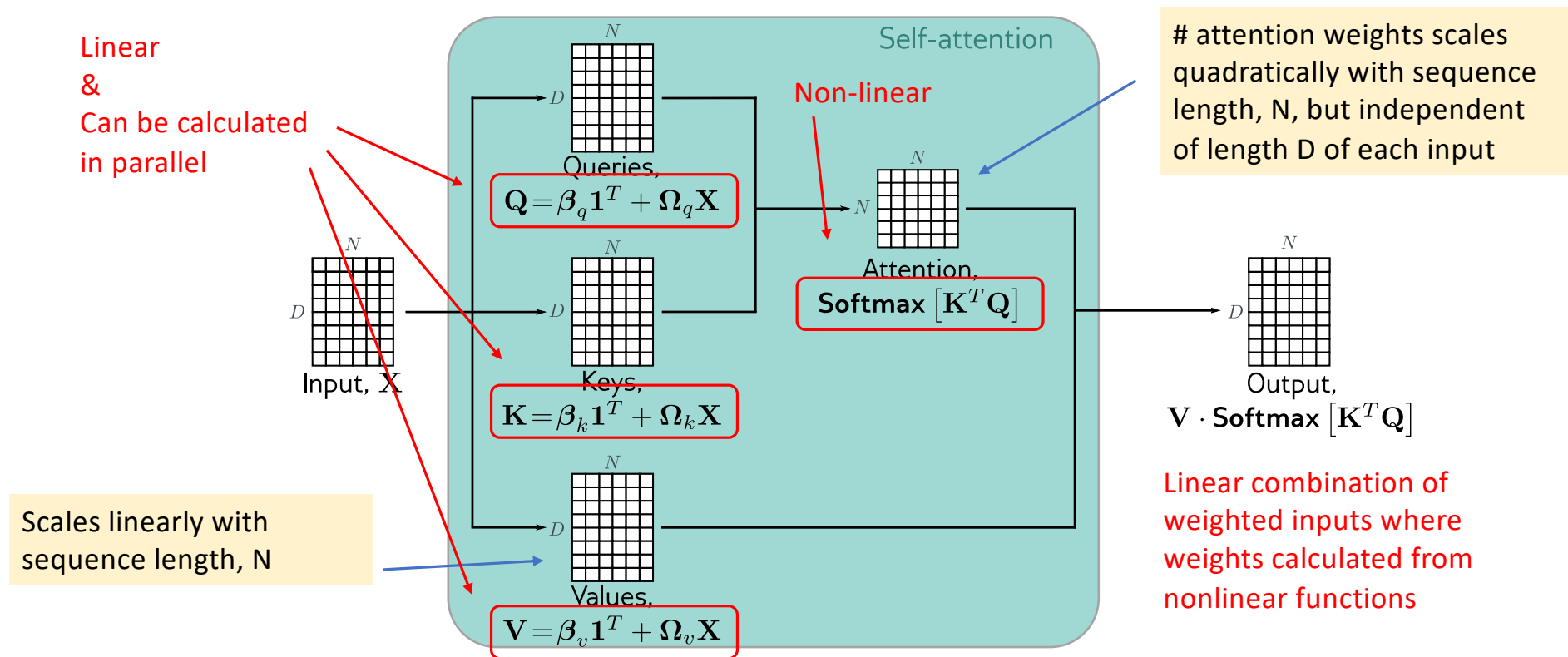
Put it all together in matrix form



Put it all together in matrix form

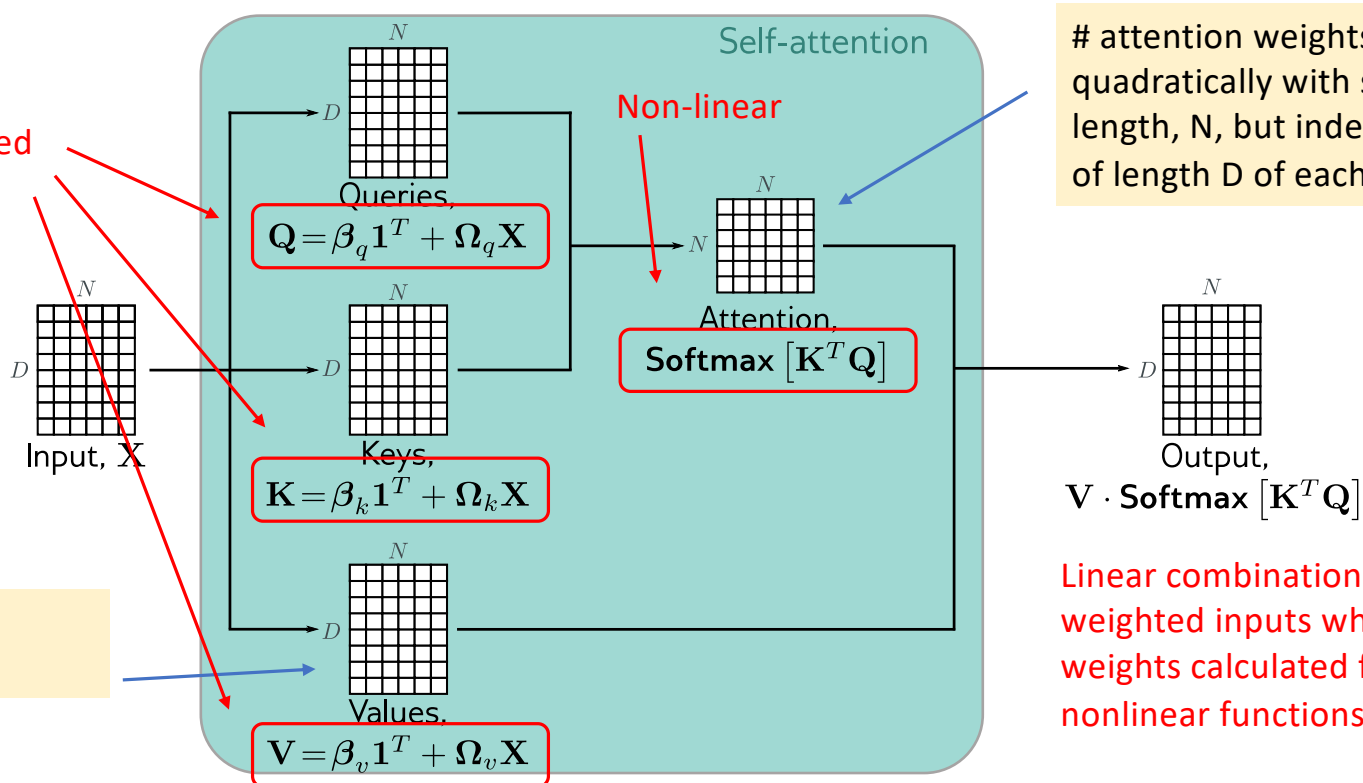


Put it all together in matrix form



Hypernetwork – 1 branch calculates weights of other branch

Linear
&
Can be calculated
in parallel

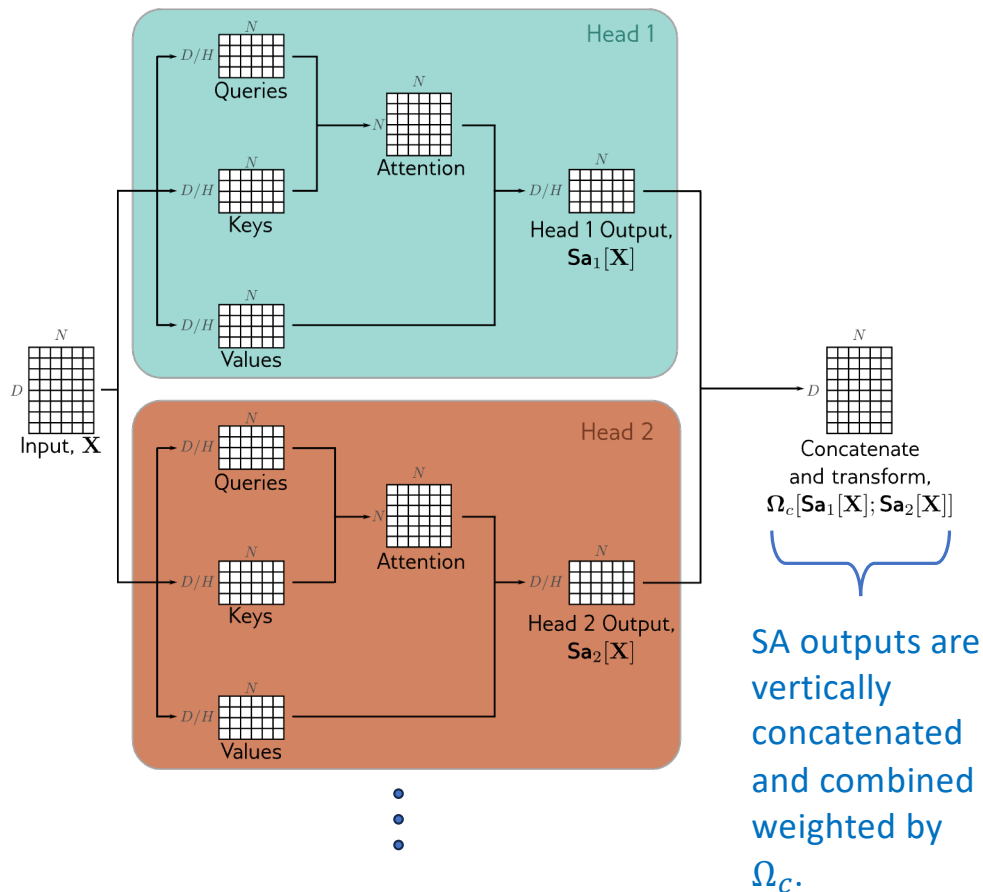


Scales linearly with
sequence length, N

attention weights scales
quadratically with sequence
length, N, but independent
of length D of each input

Linear combination of
weighted inputs where
weights calculated from
nonlinear functions

Multi-Head Self Attention



- Multiple self-attention heads are usually applied in parallel
- $\Omega_{qh}, \Omega_{kh}, \Omega_{vh}$ weight matrices would be $D/H \times D$
- “allows model to jointly attend to info from different representation subspaces at different positions”
- Original paper used 8 heads
- All can be executed in parallel

Equivariance to Word Order

A function $f[x]$ is **equivariant** to a transformation $t[]$ if: $f[t[x]] = t[f[x]]$

Self-attention is *equivariant* to permuting word order. Just a bag of words.

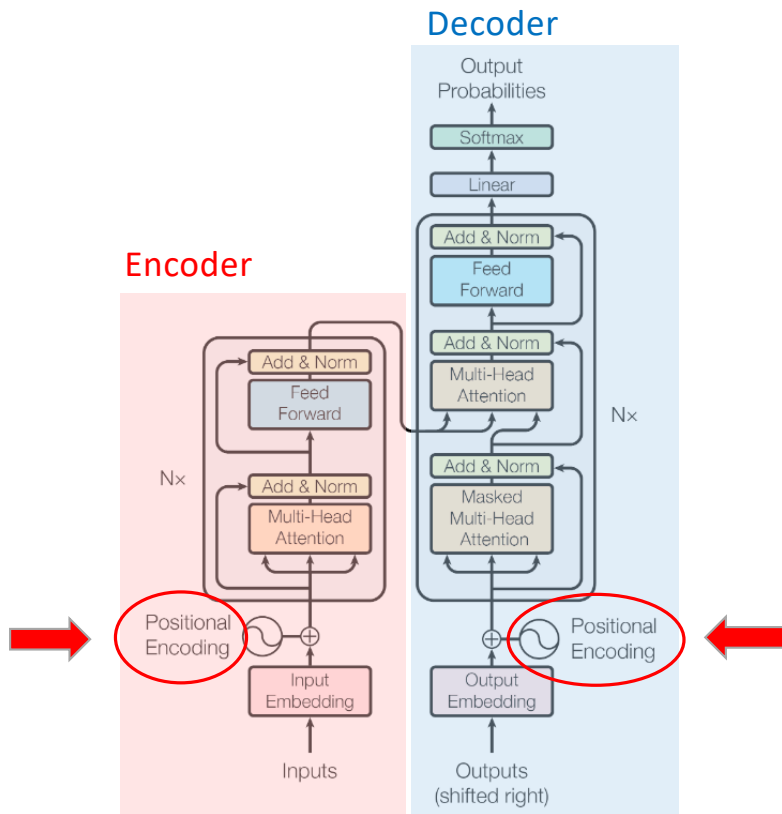
But word order is important in language:

The man ate the fish

vs.

The fish ate the man

Solution: Position Encoding



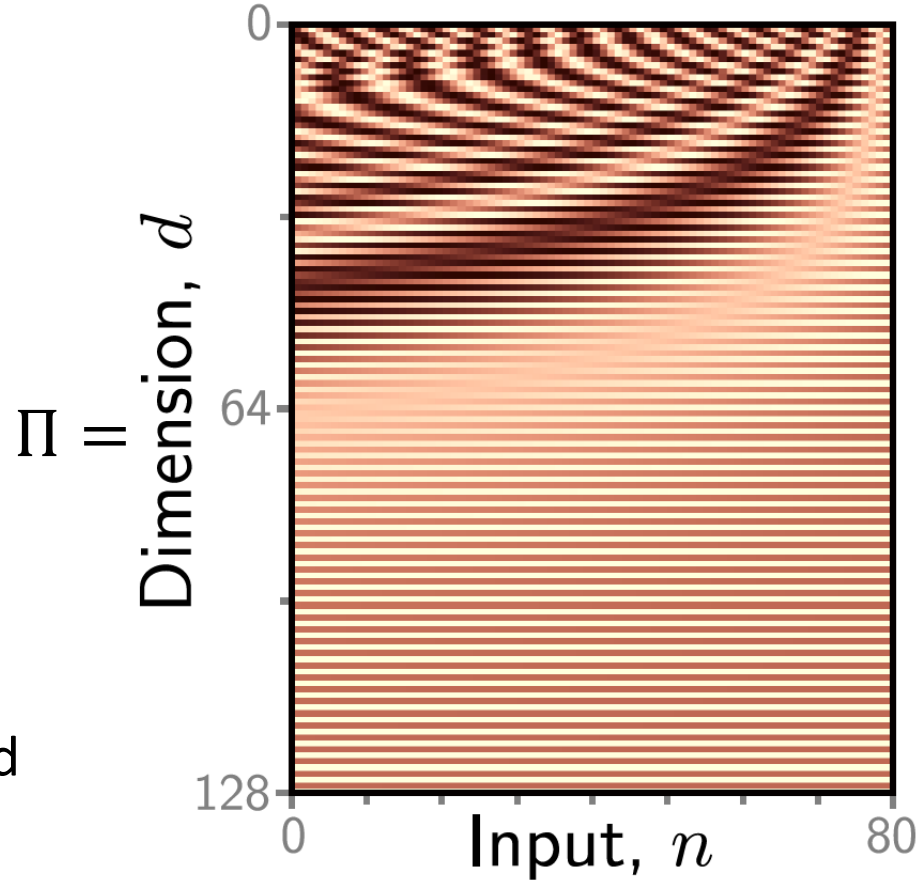
Idea is to somehow encode *absolute* or *relative* position in the inputs

Absolute Position encoding

Add some matrix, Π , to the $D \times N$ input matrix:

$$\begin{array}{c} N \\ \text{Input, } \mathbf{X} \end{array} + \Pi$$

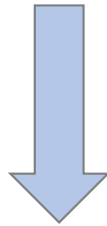
Π can be pre-defined or learned



Absolute Position encoding

Alternatively, could be added to each layer

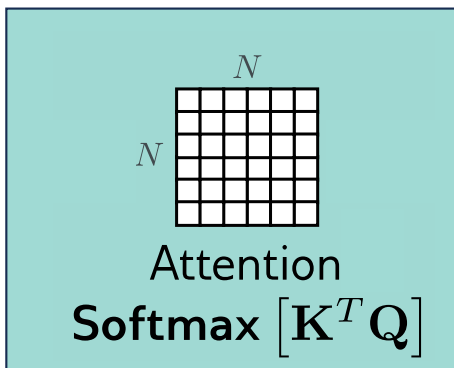
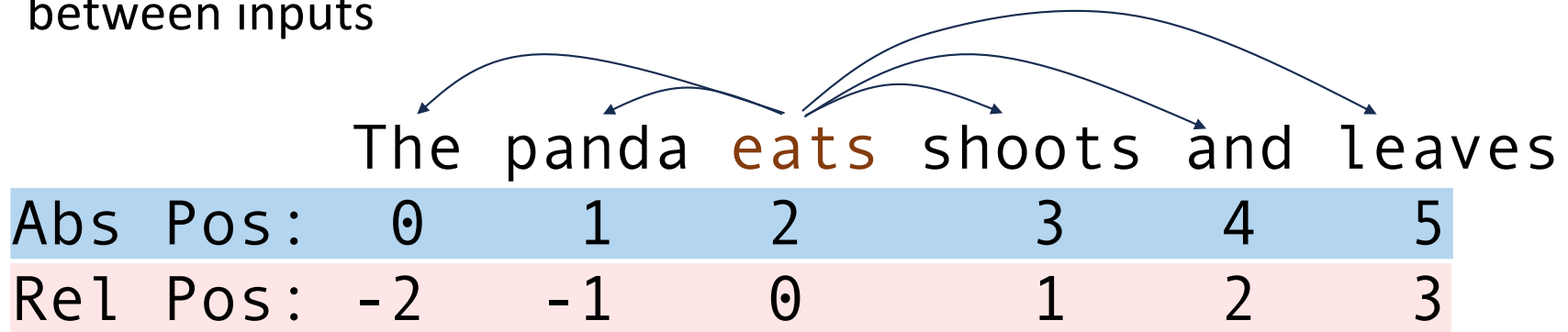
$$\mathbf{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q}]$$



$$\mathbf{Sa}[\mathbf{X}] = (\mathbf{V} + \underline{\mathbf{\Pi}}) \cdot \text{Softmax}[(\mathbf{K} + \underline{\mathbf{\Pi}})^T (\mathbf{Q} + \underline{\mathbf{\Pi}})]$$

Relative Position Encoding

Absolute position of a word is less important than relative position between inputs



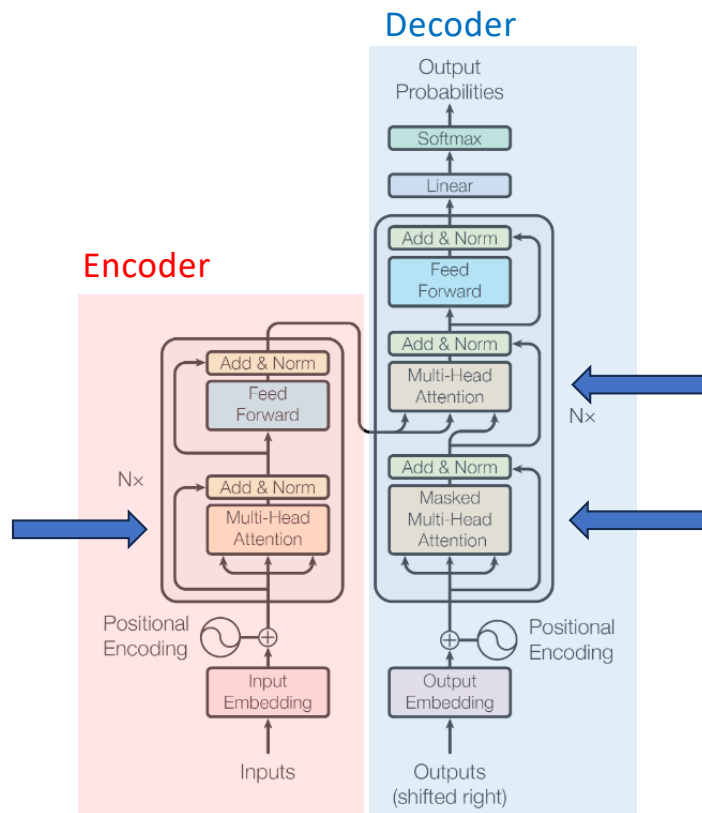
Each element of the attention matrix corresponds to an offset between query position a and key position b

Learn a parameter $\pi_{a,b}$ for each offset and modify $\text{Attention}[a,b]$ in some way.

Transformers

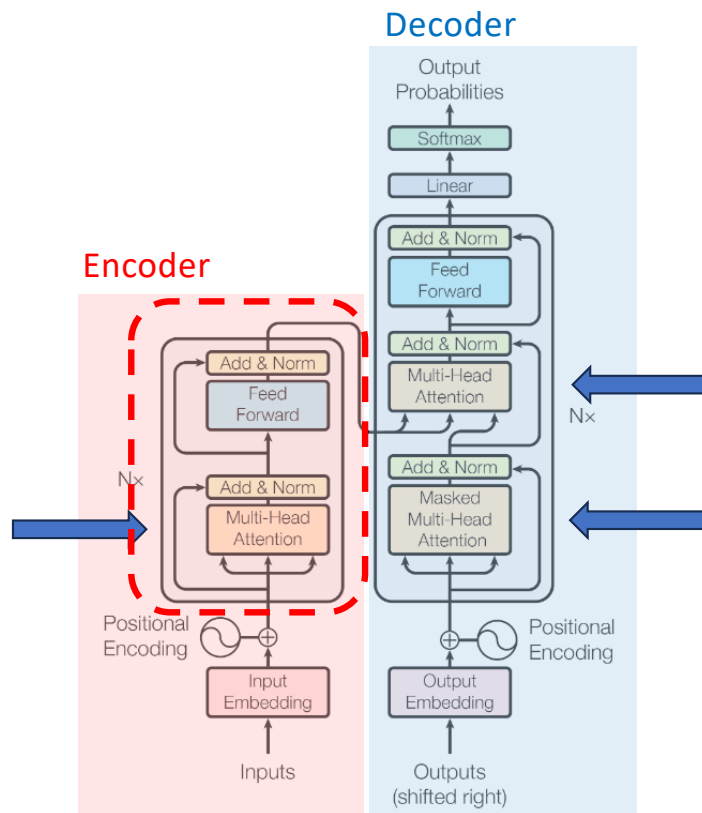
- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models

Transformers



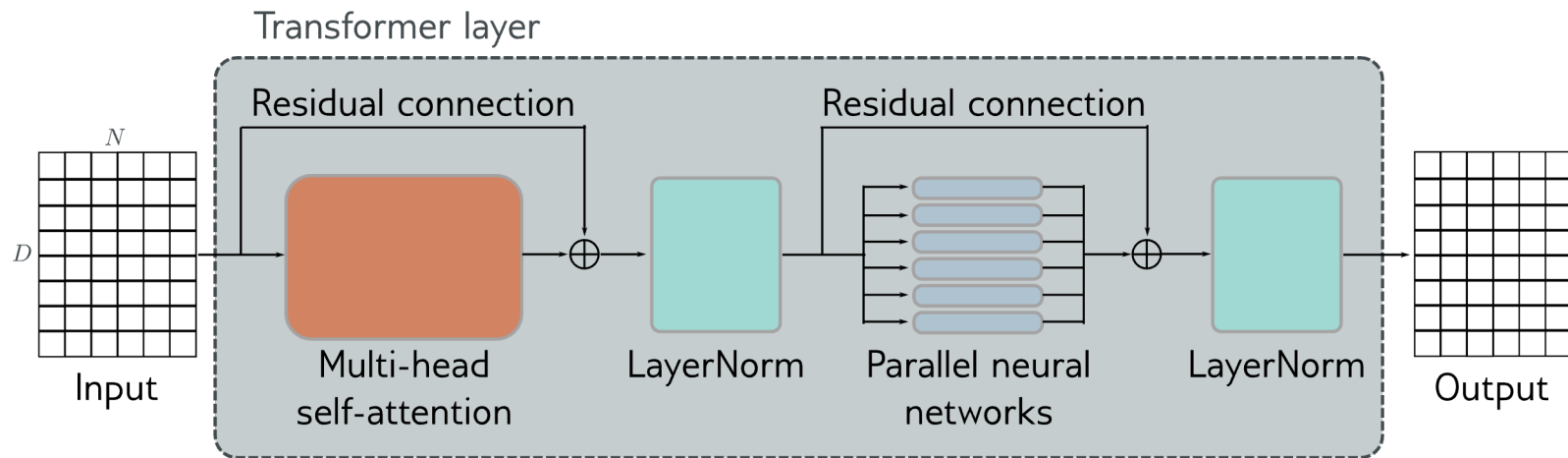
- *Multi-headed Self Attention* is just one component of the transformer architecture

Transformers



- *Multi-headed Self Attention* is just one component of the transformer architecture
- Let's look at a transformer *block* (or *layer*) from the encoder

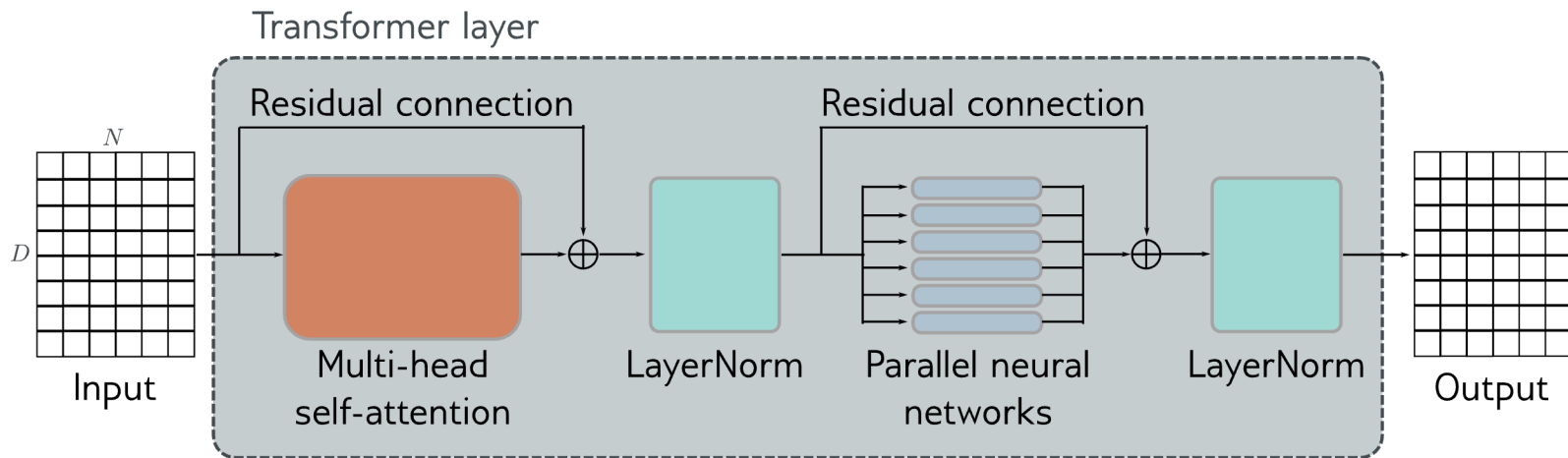
Transformer Layer -- Complete



- Adds a 2-layer MLP
- Adds residual connections around multi-head self-attentions and the parallel MLPs
- Adds LayerNorm, which normalizes across all the N input samples

Transform Layer	
$\mathbf{X}$	$\leftarrow \mathbf{X} + \text{MhSa}[\mathbf{X}]$
$\mathbf{X}$	$\leftarrow \text{LayerNorm}[\mathbf{X}]$
$\mathbf{x}_n$	$\leftarrow \mathbf{x}_n + \text{mlp}[\mathbf{x}_n]$
$\mathbf{X}$	$\leftarrow \text{LayerNorm}[\mathbf{X}],$

Transformer Layer -- MLP

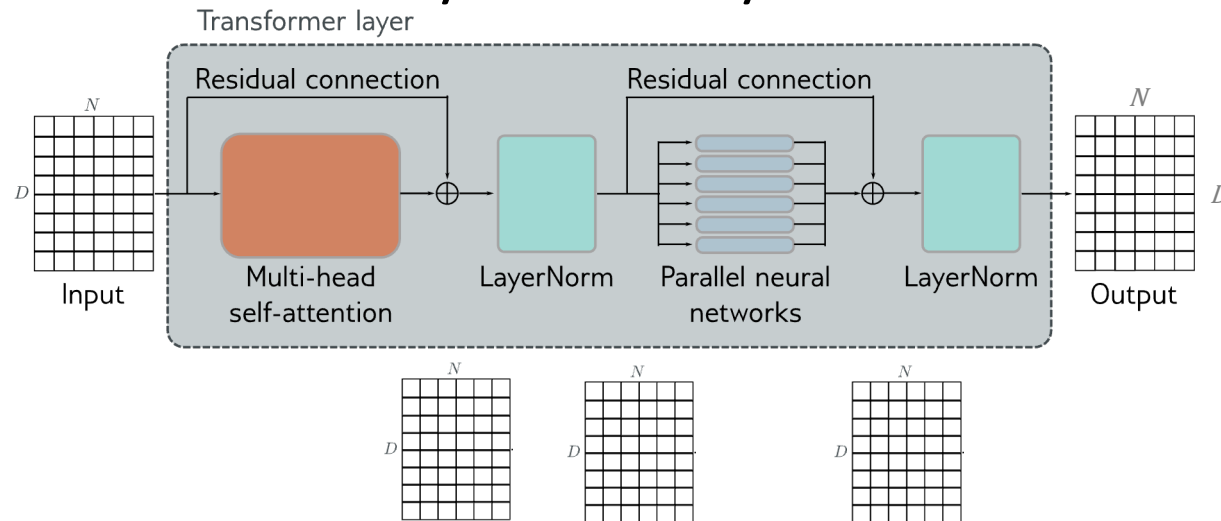


- Ads 2-layer MLP

- Same network (same weights) operates independently on each word
- Learn more complex representations and expand model capacity

$\text{Linear}_{D \times 4D} \rightarrow \text{ReLU}(\cdot) \rightarrow \text{Linear}_{4D \times D}$

Transformer Layer -- LayerNorm



- Normalize across same layer
- Learned gain and offset

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Calculated column-wise

NLP Example

```
batch, sentence_length, embedding_dim = 20, 5, 10  
embedding = torch.randn(batch, sentence_length, embedding_dim)  
layer_norm = nn.LayerNorm(embedding_dim)
```

Activate module

```
layer_norm(embedding)
```

<https://pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html>

Transformers

- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models

Transformers

- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models
 - Encoder
 - Decoder
 - Encoder-Decoder

Transformers

- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models
 - Encoder
 - Decoder
 - Encoder-Decoder

3 Types of Transformer Models

1. *Encoder* – transforms text embeddings into representations that support variety of tasks (e.g. sentiment analysis, classification)
❖ Model Example: BERT
2. *Decoder* – predicts the next token to continue the input text (e.g. ChatGPT, AI assistants)
❖ Model Example: GPT4o
3. *Encoder-Decoder* – used in sequence-to-sequence tasks, where one text string is converted to another (e.g. machine translation)

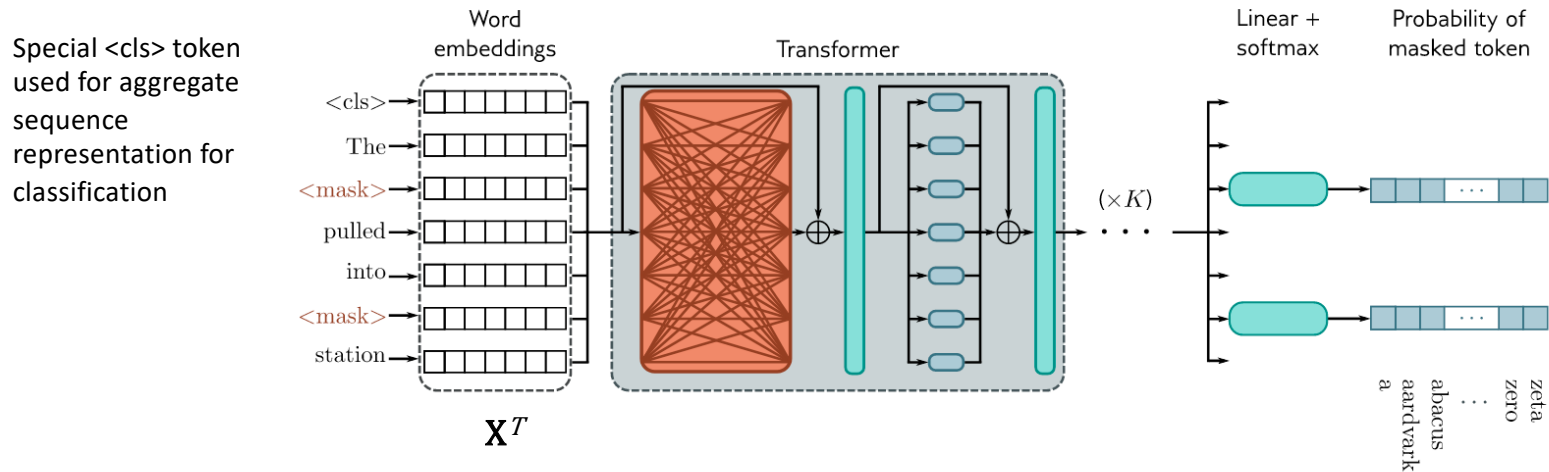
Encoder Model Example: BERT (2019)

Bidirectional Encoder Representations from Transformers

- Hyperparameters
 - 30,000 token vocabulary
 - 1024-dimensional word embeddings
 - 24x transformer layers
 - 16 heads in self-attention mechanism
 - 4096 hidden units in middle of MLP
- ~340 million parameters
- *Pre-trained* in a *self-supervised* manner,
- then can be adapted to task with one additional layer and *fine-tuned*

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." arXiv, May 24, 2019. doi: [10.48550/arXiv.1810.04805](https://doi.org/10.48550/arXiv.1810.04805).

Encoder Pre-Training

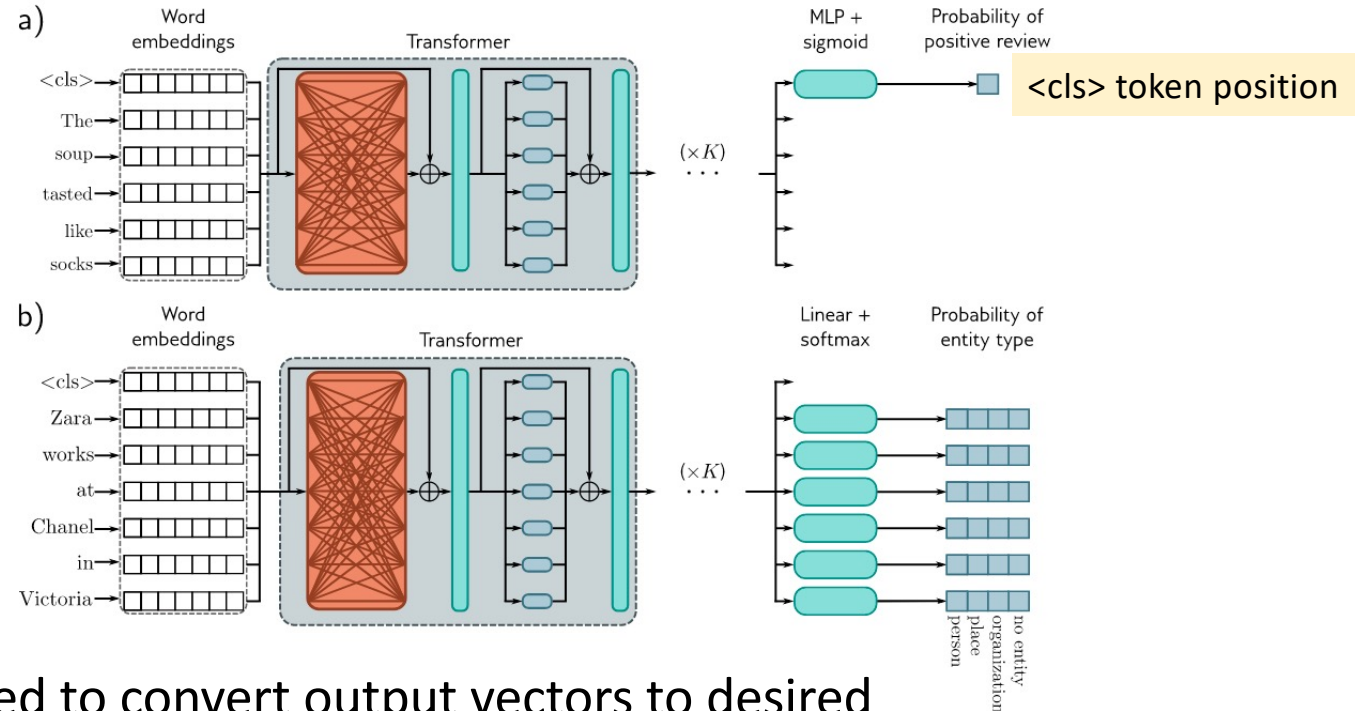


- A small percentage of input embedding replaced with a generic <mask> token
- Predict missing token from output embeddings
- Added linear layer and softmax to generate probabilities over vocabulary
- Trained on BooksCorpus (800M words) and English Wikipedia (2.5B words)

Encoder Fine-Tuning

Sentiment Analysis

Named Entity Recognition (NER)



- Extra layer(s) appended to convert output vectors to desired output format
- 3rd Example: Text span prediction -- predict start and end location of answer to a question in passage of Wikipedia, see <https://rajpurkar.github.io/SQuAD-explorer/>

Transformers

- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models
 - Encoder
 - Decoder
 - Encoder-Decoder

Decoder Model Example: GPT3 (2020)

Generative Pre-trained Transformer

- One purpose: *generate the next token in a sequence*
- By constructing an autoregressive model

Decoder Model Example: GPT3 (2020)

Generative Pre-trained Transformer

- One purpose: *generate the next token in a sequence*
- By constructing an autoregressive model
- Factors the probability of the sentence:

$$\begin{aligned} \Pr(\textit{Learning deep learning is fun}) = & \\ & \Pr(\textit{Learning}) \times \Pr(\textit{deep} \mid \textit{learning}) \times \\ & \Pr(\textit{learning} \mid \textit{Learning deep}) \times \\ & \Pr(\textit{is} \mid \textit{Learning deep learning}) \times \\ & \Pr(\textit{fun} \mid \textit{Learning deep learning is}) \end{aligned}$$

Decoder Model Example: GPT3 (2020)

Generative Pre-trained Transformer

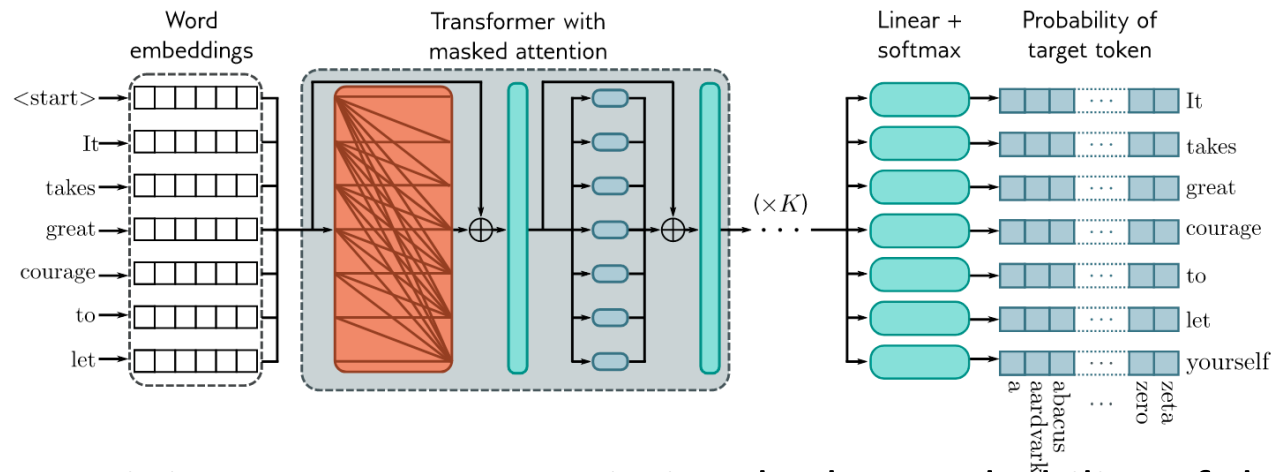
- One purpose: *generate the next token in a sequence*
- By constructing an autoregressive model
- Factors the probability of the sentence:

$$\begin{aligned} \Pr(\textit{Learning deep learning is fun}) = & \\ & \Pr(\textit{Learning}) \times \Pr(\textit{deep} \mid \textit{learning}) \times \\ & \Pr(\textit{learning} \mid \textit{Learning deep}) \times \\ & \Pr(\textit{is} \mid \textit{Learning deep learning}) \times \\ & \Pr(\textit{fun} \mid \textit{Learning deep learning is}) \end{aligned}$$

- More formally: Autoregressive model_N

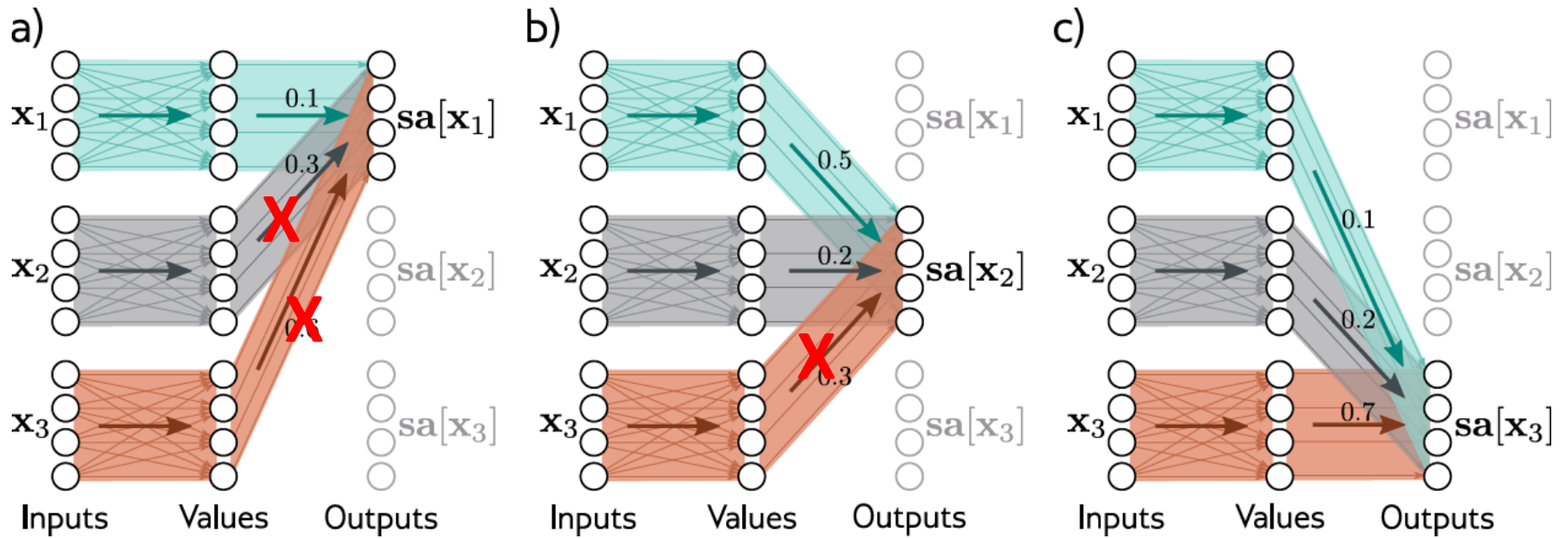
$$\Pr(t_1, t_2, \dots, t_N) = \Pr(t_1) \prod_{n=2}^N \Pr(t_n \mid t_1, t_2, \dots, t_{n-1})$$

Decoder: *Masked* Self-Attention



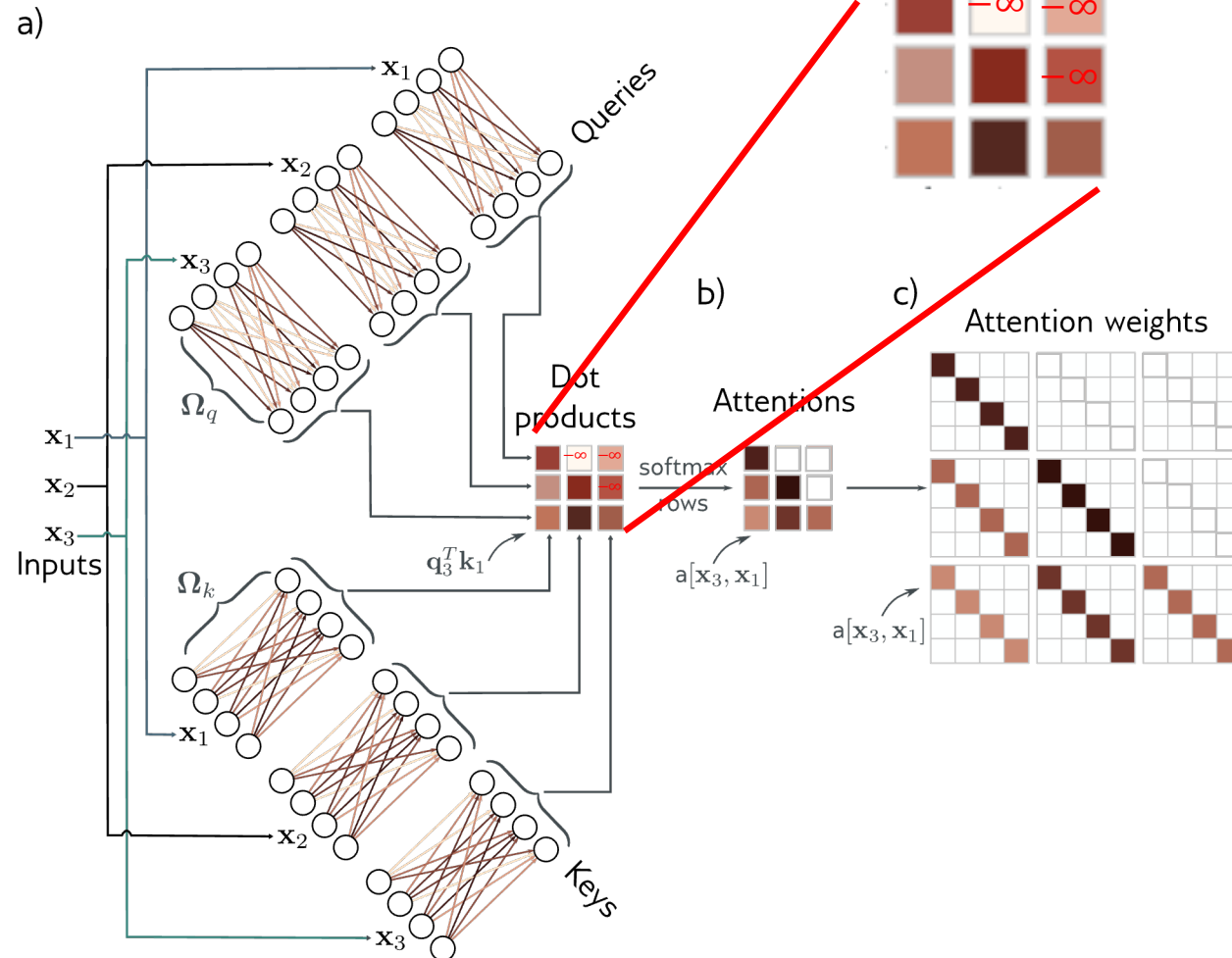
- During training we want to maximize the log probability of the input text under the autoregressive model
- We want to make sure the model doesn't “cheat” during training by looking ahead at the next token
- Hence we mask the self attention weights corresponding to current and right context to *negative infinity*

Masked Self-Attention

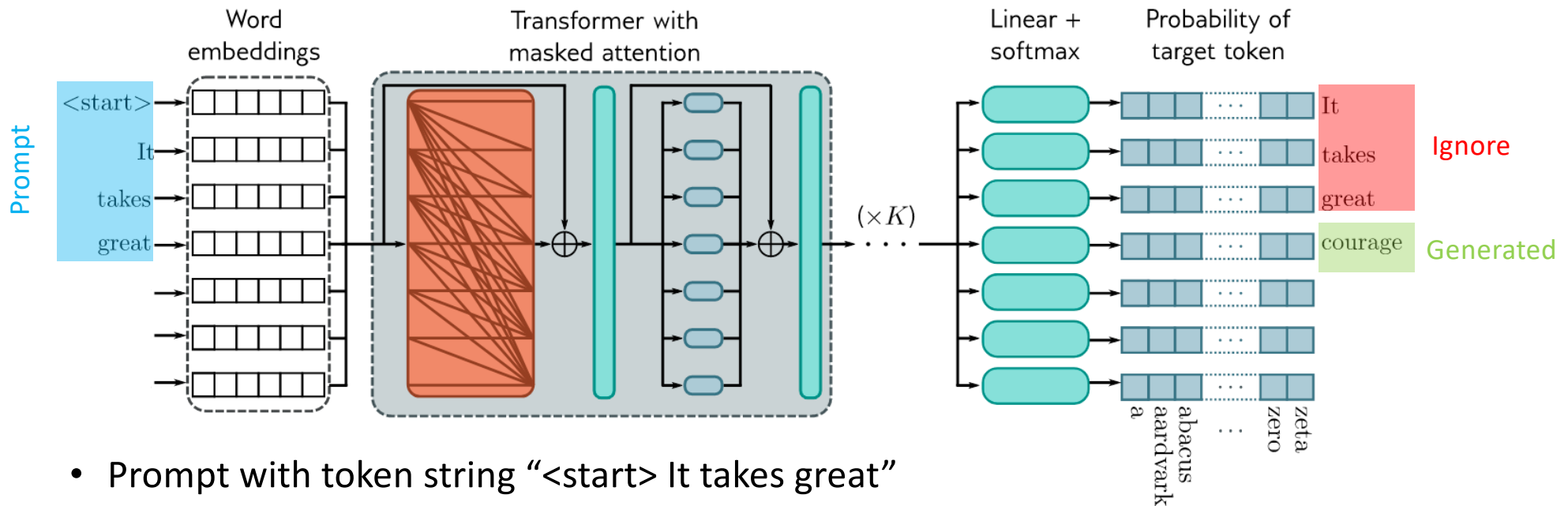


Mask right context self-attention weights to zero

Masked Self-Attention

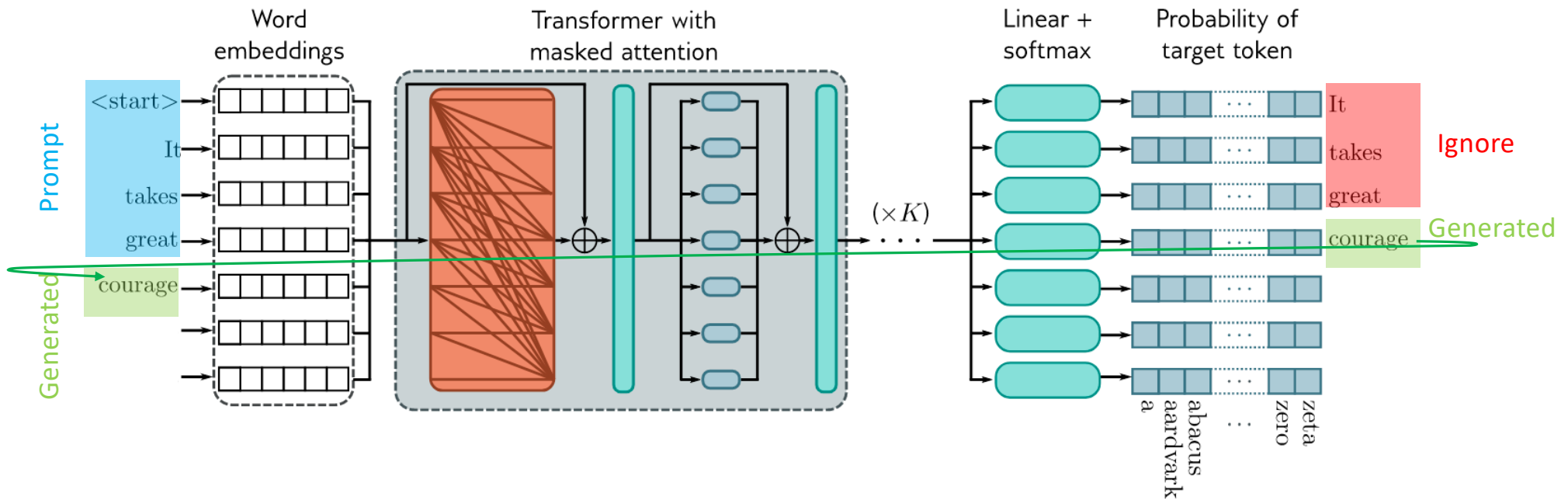


Decoder: Text Generation (Generative AI)



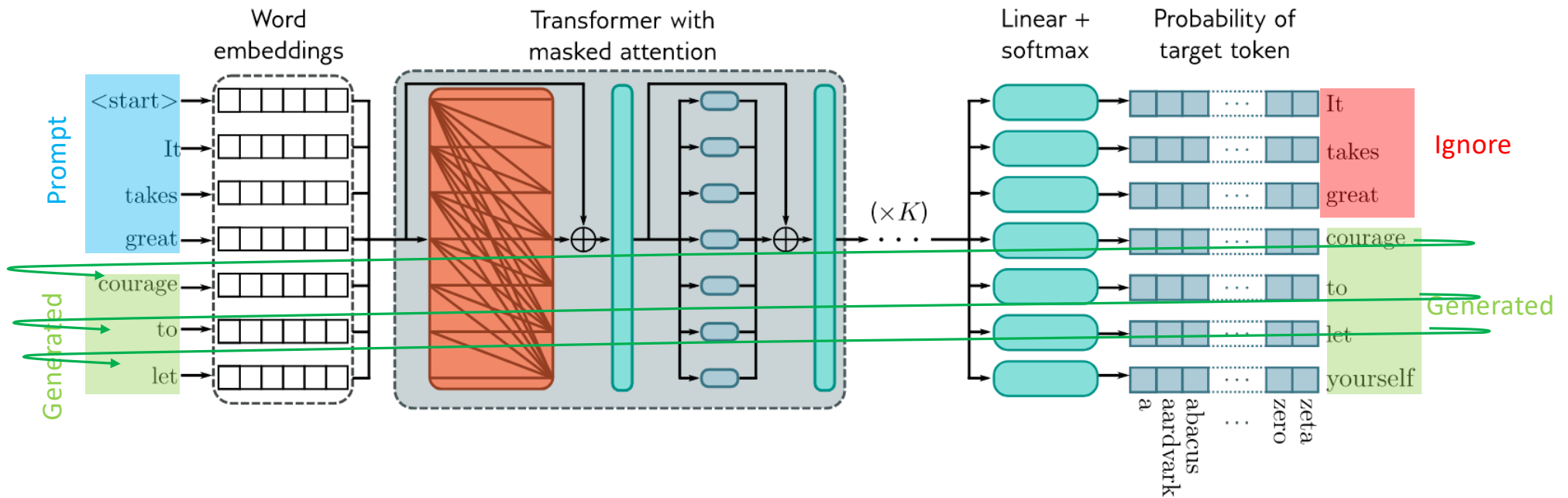
- Prompt with token string "<start> It takes great"
- Generate next token for the sequence by
 - picking most likely token
 - sample from the probability distribution
 - alternative *top-k* sampling to avoid picking from the long tail
 - beam search – select the most likely sentence rather than greedily pick

Decoder: Text Generation (Generative AI)



- Feed the output back into input

Decoder: Text Generation (Generative AI)



- Feed the output back into input

Technical Details

	BERT	GPT3
Model Architecture	Encoder	Decoder
Embedding Size	1024	12,288
Vocabulary	30K tokens	
Sequence Length		2048
# Heads	16	96
# Layers	24	96
Q,K,V dimensions	64	128
Training set size	3.3B tokens	300B+ tokens
# Parameters	340M	175B

Transformers

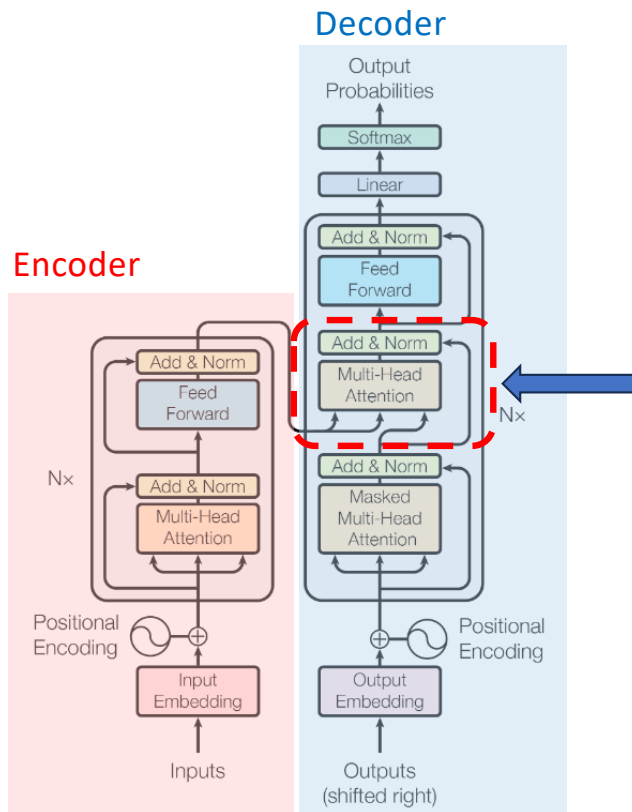
- Motivation
- Dot-product self-attention
- Applying Self-Attention
- The Transformer Architecture
- Three Types of NLP Transformer Models
 - Encoder
 - Decoder
 - Encoder-Decoder

Encoder-Decoder Model

- Used for *machine translation*, which is a *sequence-to-sequence* task



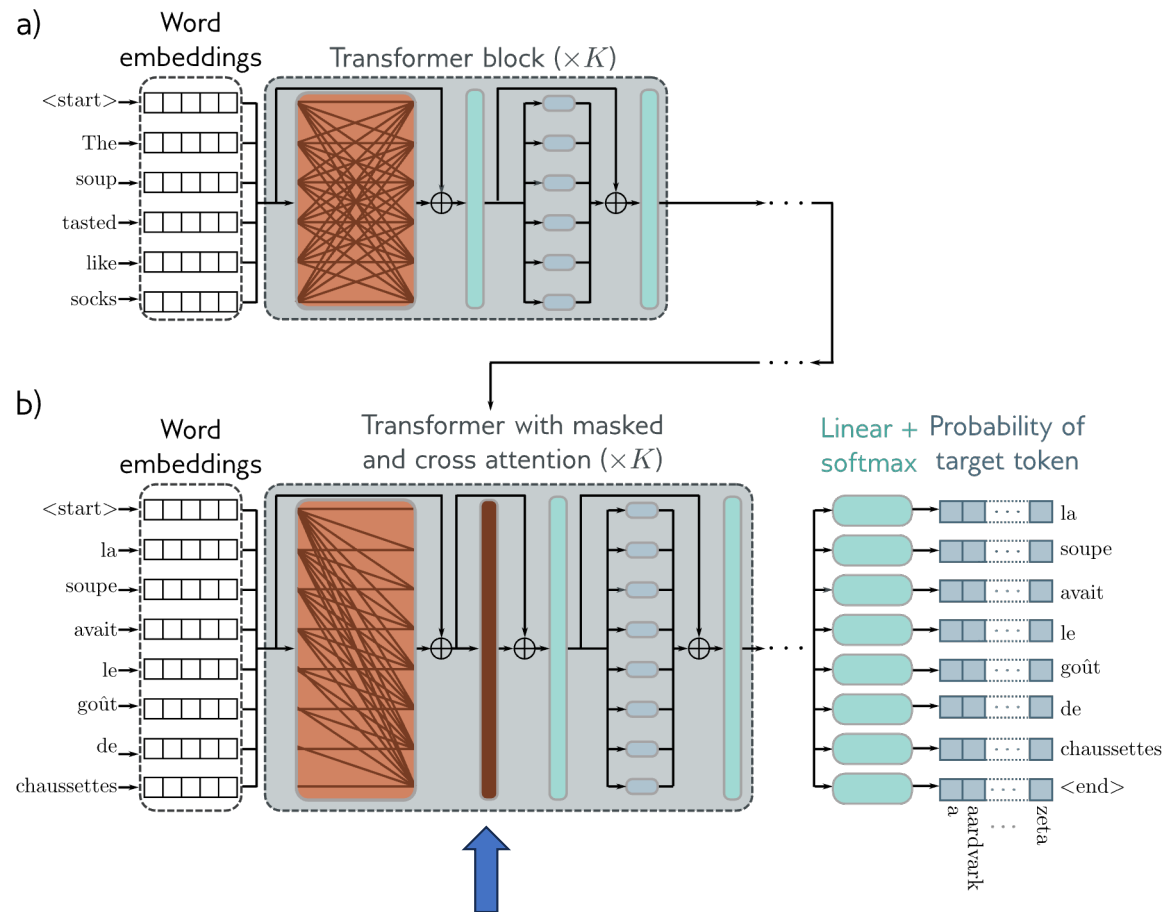
Encoder Decoder Model



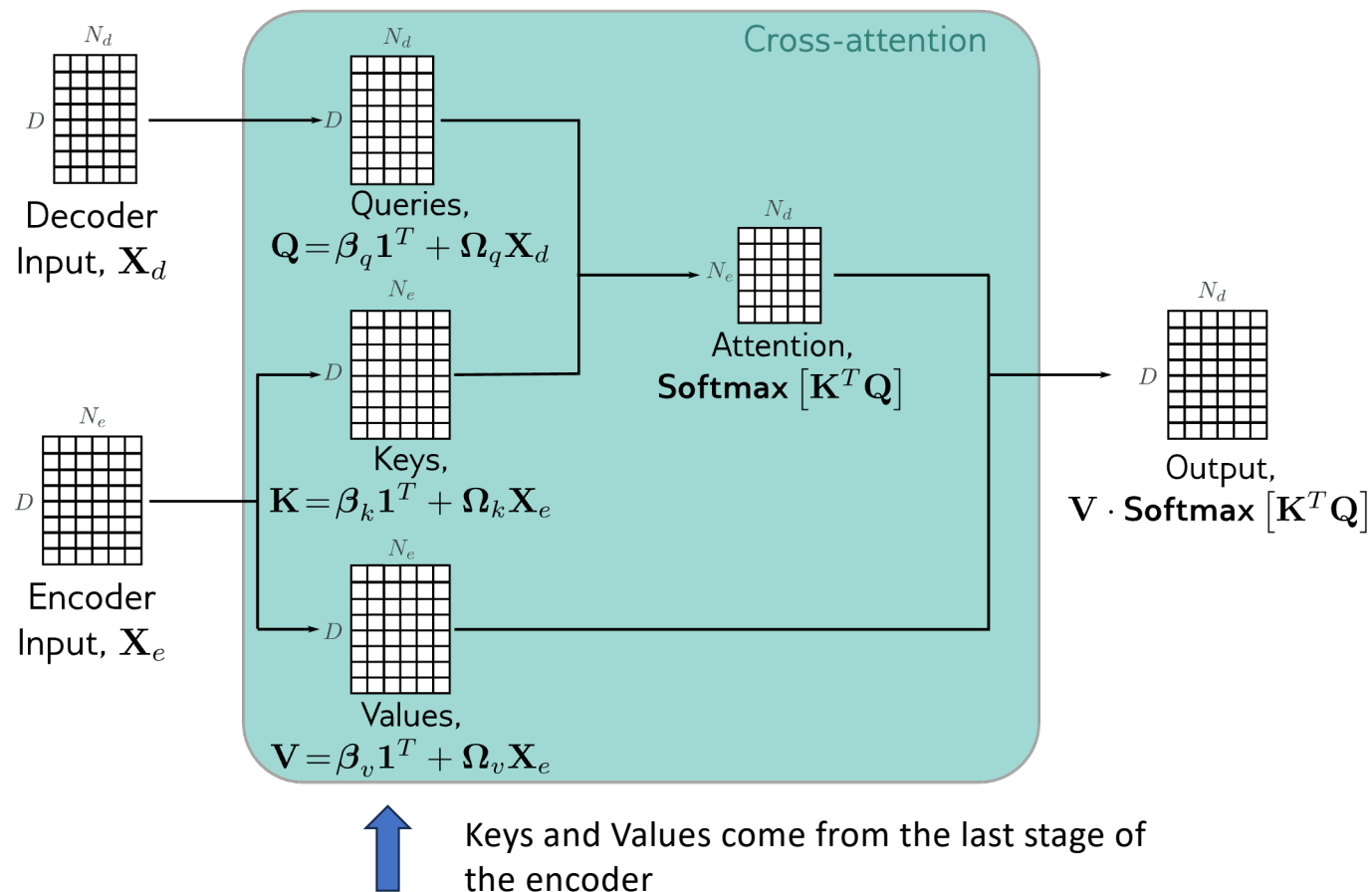
- The transformer layer in the decoder of the encoder-decoder model has an extra stage
- Attends to the input of the encoder with *cross attention* using Keys and Values from the output of the encoder
- Shown here on original diagram from “Attention is all you need” paper

Encoder Decoder Model

- Same view per UDL book



Cross-Attention



slido



Which model flavor do you use for Named Entity Recognition?

① Start presenting to display the poll results on this slide.

slido



Which model flavor do you use for language translation?

① Start presenting to display the poll results on this slide.

slido



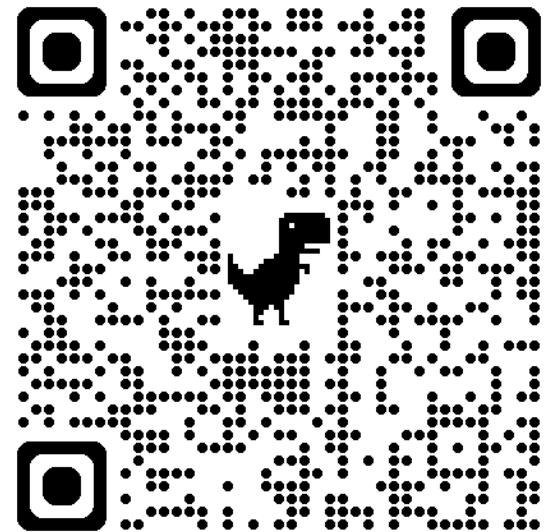
Which model flavor do you use for generating text, question answering, AI assistant?

① Start presenting to display the poll results on this slide.

Next Time

- Tokenization and Learned Embeddings
- Training and Fine-Tuning Transformers
- Image Transformers
- Multimodal Transformers
- ...

Feedback



<https://forms.gle/pXHM5nx1Ti9aFmpw6>